

JavaScript

Objective -

- Introduction
- Why to learn JavaScript?
- Sample Program
- Applications of JavaScript
- Advantages of JavaScript
- Limitations of JavaScript

Introduction -

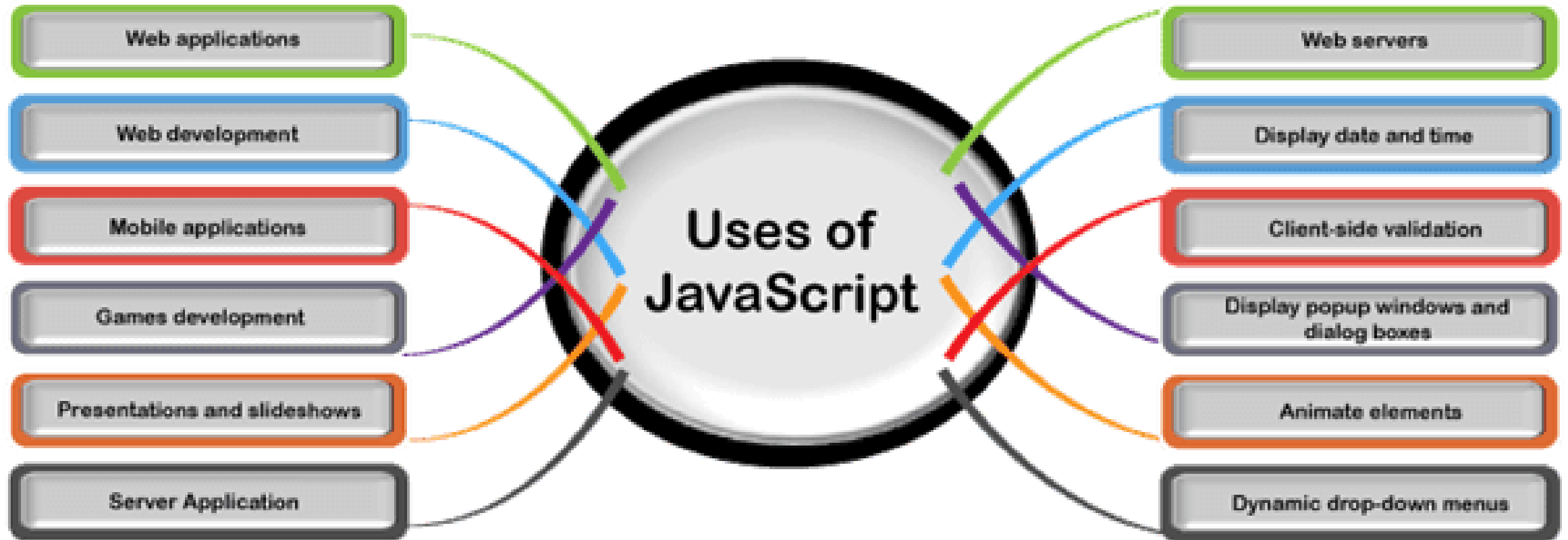
- JavaScript is a dynamic computer programming language. It is lightweight and most commonly used as a part of web pages, whose implementations allow client-side script to interact with the user and make dynamic pages
- It is an interpreted programming language with object-oriented capabilities
- It is first implemented by Netscape (with help from Sun Microsystems). JavaScript was created by Brendan Eich at Netscape in 1995 for the purpose of allowing code in web-pages (performing logical operation on client side)

Why to learn JavaScript?

- Javascript is the most popular programming language in the world and that makes it a programmer's great choice. Once you learnt Javascript, it helps you developing great front-end as well as back-end softwares using different Javascript based frameworks like jQuery, Node.JS etc
- Javascript is everywhere, it comes installed on every modern web browser and so to learn Javascript you really do not need any special environment setup
- Javascript helps you create really beautiful and crazy fast websites. You can develop your website with a console like look and feel and give your users the best Graphical User Experience.
- JavaScript usage has now extended to mobile app development, desktop app development, and game development. This opens many opportunities for you as Javascript Programmer.

- There are many useful **Javascript frameworks** and libraries available:
- Angular
- React
- jQuery
- Vue.js

Applications of JavaScript -



Advantages of JavaScript -

- The merits of using JavaScript are –
- **Less server interaction** – You can validate user input before sending the page off to the server. This saves server traffic, which means less load on your server.
- **Immediate feedback to the visitors** – They don't have to wait for a page reload to see if they have forgotten to enter something.
- **Increased interactivity** – You can create interfaces that react when the user hovers over them with a mouse or activates them via the keyboard.

Limitations of JavaScript -

We cannot treat JavaScript as a full-fledged programming language. It lacks the following important features –

- Client-side JavaScript does not allow the reading or writing of files. This has been kept for security reason.
- JavaScript cannot be used for networking applications because there is no such support available.
- JavaScript doesn't have any multi-threading or multiprocessor capabilities.

JavaScript Syntax -

- JavaScript can be implemented using JavaScript statements that are placed within the **<script>... </script>** HTML tags in a web page.
- You can place the **<script>** tags, containing your JavaScript, anywhere within your web page, but it is normally recommended that you should keep it within the **<head>** tags.
- The **<script>** tag alerts the browser program to start interpreting all the text between these tags as a script. A simple syntax of your JavaScript will appear as follows.

```
<script ...>  
    JavaScript code  
</script>
```

JavaScript Syntax -

- There is a flexibility given to include JavaScript code anywhere in an HTML document. However the most preferred ways to include JavaScript in an HTML file are as follows –
- Script in `<head>...</head>` section.
- Script in `<body>...</body>` section.
- Script in `<body>...</body>` and `<head>...</head>` sections.
- Script in an external file and then include in `<head>...</head>` section.

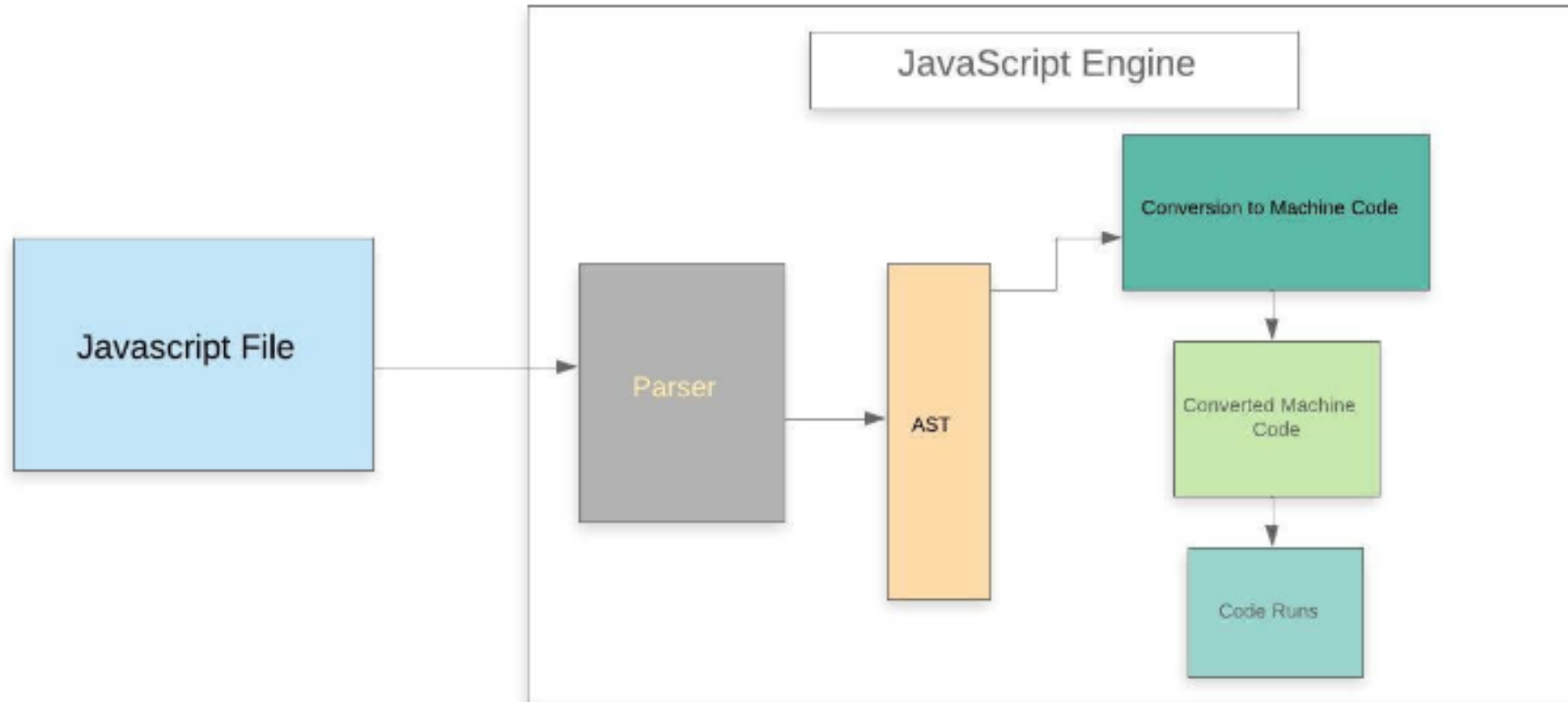
Sample program -

```
<!DOCTYPE html>
<html lang="en">
<head>
  <meta charset="UTF-8">
  <meta http-equiv="X-UA-Compatible" content="IE=edge">
  <meta name="viewport" content="width=device-width, initial-
scale=1.0">
  <title>Document</title>
  <script>

    document.write("This is my first java script program");

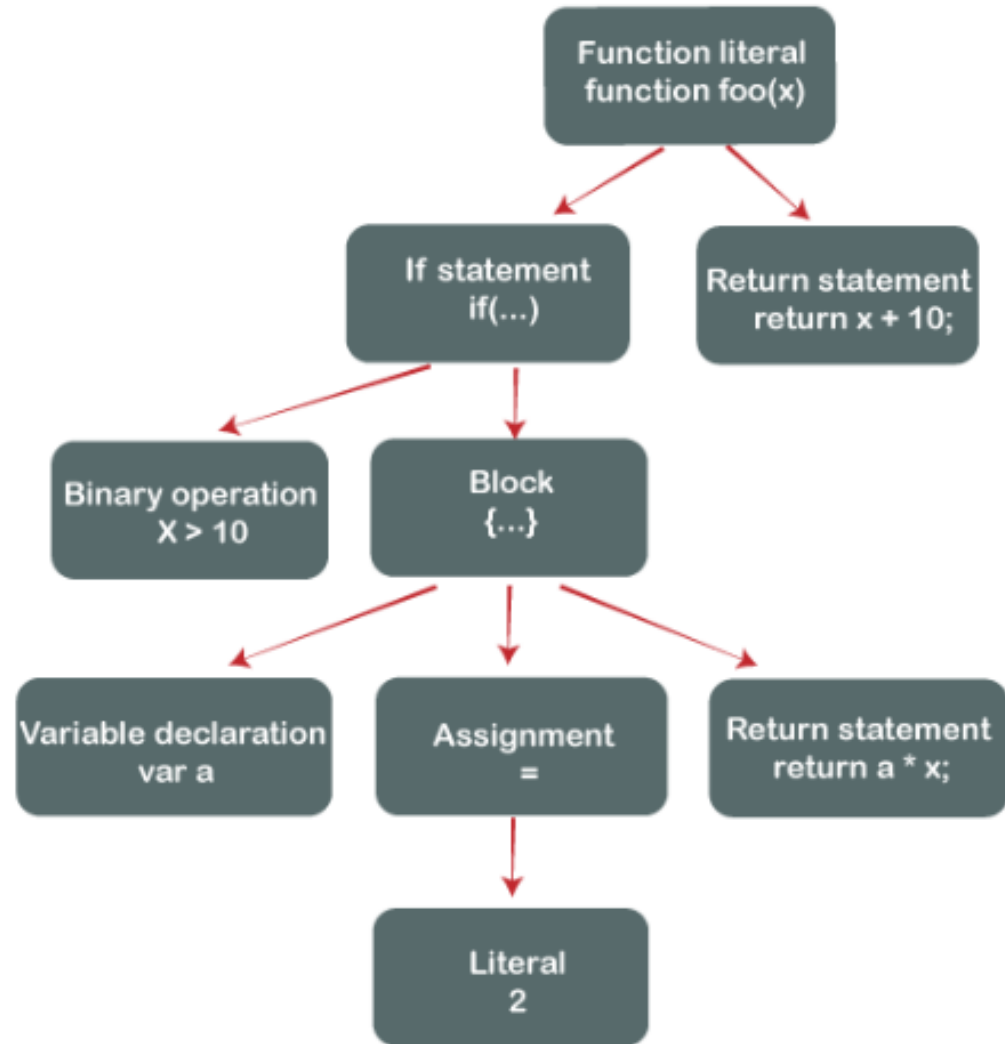
  </script>
</head>
<body>
</body>
</html>
```

Working of JavaScript-

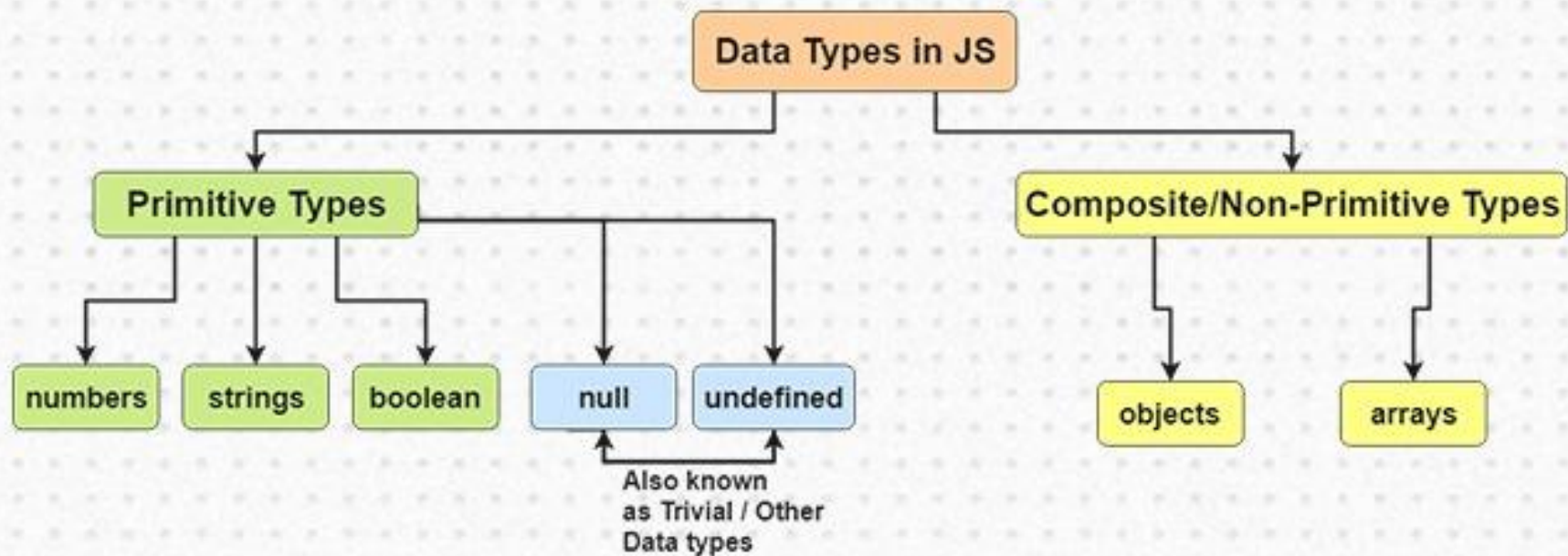


Understanding AST-

```
function fun(x) {  
  if (x > 15) {  
    var a = 4;  
    return a * x;  
  }  
  
  return x + 10;  
}
```



Variables & Datatypes ***in JavaScript***



JavaScript Variables-

- Variables can be thought of as named containers. You can place data into these containers and then refer to the data simply by naming the container.
- Before you use a variable in a JavaScript program, you must declare it. Variables are declared with the **var** keyword as follows.

You can also declare multiple variables with the same **var** keyword as follows –

```
<script type = "text/javascript">  
var money;  
var name;  
</script>
```

```
<script type = "text/javascript">  
var money, name;  
</script>
```

Variable initialization -

Storing a value in a variable is called **variable initialization**. You can do variable initialization at the time of variable creation or at a later point in time when you need that variable

Note – Use the **var** keyword only for declaration or initialization, once for the life of any variable name in a document. You should not re-declare same variable twice

JavaScript is **untyped** language. This means that a JavaScript variable can hold a value of any data type. Unlike many other languages, you don't have to tell JavaScript during variable declaration what type of value the variable will hold. The value type of a variable can change during the execution of a program and JavaScript takes care of it automatically

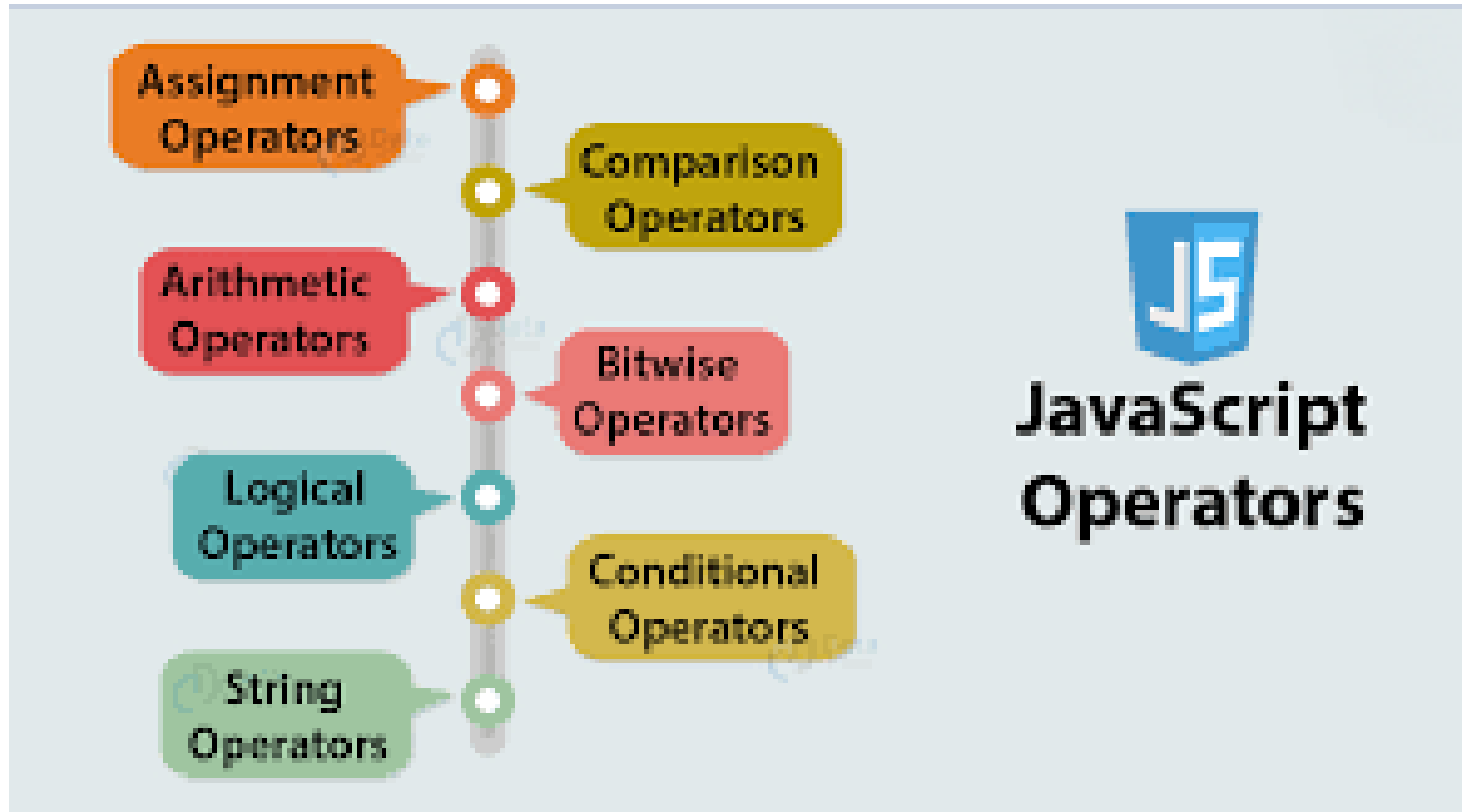
```
<script type = "text/javascript">  
  var name = "Rohit";  
  var money;  
  money = 2000.50;  
</script>
```


JavaScript variable scope -

- The scope of a variable is the region of your program in which it is defined. JavaScript variables have only two scopes
- **Global Variables** – A global variable has global scope which means it can be defined anywhere in your JavaScript code.
- **Local Variables** – A local variable will be visible only within a function where it is defined. Function parameters are always local to that function.

Within the body of a function, a local variable takes precedence over a global variable with the same name. If you declare a local variable or function parameter with the same name as a global variable, you effectively hide the global variable.

JavaScript Operators-



Arithmetic Operators-

JavaScript supports the following arithmetic operators –

Assume variable A holds 10 and variable B holds 20, then –

Note – Addition operator (+) works for Numeric as well as Strings. e.g. "a" + 10 will give "a10".

Sr.No.	Operator & Description
1	+ (Addition) Adds two operands Ex: A + B will give 30
2	- (Subtraction) Subtracts the second operand from the first Ex: A - B will give -10
3	* (Multiplication) Multiply both operands Ex: A * B will give 200
4	/ (Division) Divide the numerator by the denominator Ex: B / A will give 2
5	% (Modulus) Outputs the remainder of an integer division Ex: B % A will give 0
6	++ (Increment) Increases an integer value by one Ex: A++ will give 11
7	-- (Decrement) Decreases an integer value by one Ex: A-- will give 9

Arithmetic Operators Example-

```
<!DOCTYPE html>
<html lang="en">
<head>
  <meta charset="UTF-8">
  <meta http-equiv="X-UA-Compatible" content="IE=edge">
  <meta name="viewport" content="width=device-width, initial-scale=1.0">
  <title>Document</title>
  <script>

    var a = 33;
    var b = 10;
    var c = "Test";
    var newline = "<br />";

    document.write("a + b = ");
    result = a + b;
    document.write(result);
    document.write(newline);

    document.write("a - b = ");
    result = a - b;
    document.write(result);
    document.write(newline);

    document.write("a / b = ");
    result = a / b;
    document.write(result);
    document.write(newline);

    document.write("a % b = ");
    result = a % b;
    document.write(result);
    document.write(newline);

    document.write("a + b + c = ");
    result = a + b + c;
    document.write(result);
    document.write(newline);

    a = ++a;
    document.write("++a = ");
    result = ++a;
    document.write(result);
    document.write(newline);

    b = --b;
    document.write("--b = ");
    result = --b;
    document.write(result);
    document.write(newline);

    var i=10;
    document.write(--i);

  </script>
</head>
```

Logical Operators Example-

JavaScript supports the following logical operators –
Assume variable A holds 10
and variable B holds 20, then –

Sr.No.	Operator & Description
1	&& (Logical AND) If both the operands are non-zero, then the condition becomes true. Ex: (A && B) is true.
2	 (Logical OR) If any of the two operands are non-zero, then the condition becomes true. Ex: (A B) is true.
3	! (Logical NOT) Reverses the logical state of its operand. If a condition is true, then the Logical NOT operator will make it false. Ex: ! (A && B) is false.

1f. Use `document.body.innerHTML = ...;` to display 'Good morning!' on the web page.

1g. Display your name on the web page.

Challenge Exercises

1h. You order a T-shirt for \$10, socks for \$8, and dinner plates for \$20. Use JavaScript to calculate the total cost of your order.

1i. Your bank account has \$100, you spend \$20 on lunch, \$50 on dinner, and earn \$200 from your job. Calculate how much money you have.

1j. Use `document.body.innerHTML = ...;` to make the web page blank.

Logical Operators -

Logical AND Operator -

&& evaluates to true if both the operands are true, else evaluates to false

Note: You can also use logical operators with numbers. In JavaScript, 0 is false and all non-zero values are true.

Logical OR Operator -

// evaluates to true if either of the operands is true. If both operands are false, the result is false.

Logical NOT Operator -

! evaluates to true if the operand is false and vice-versa.

Logical Operators Example-

```
<!DOCTYPE html>
<html lang="en">
<head>
  <meta charset="UTF-8">
  <meta http-equiv="X-UA-Compatible" content="IE=edge">
  <meta name="viewport" content="width=device-width, initial-
scale=1.0">
  <title>Document</title>
  <script>
    var a = true;
    var b = false;
    var result;
    var newline = "<br />";

    document.write("(a && b) => ");
    result = (a && b);
    document.write(result);
    document.write(newline);

    document.write("(a || b) => ");
    result = (a || b);
    document.write(result);
    document.write(newline);

    document.write("(!(a && b) => ");
    result = (!(a && b));
    document.write(result);
    document.write(newline);

  </script>
</head>
<body>

</body>
</html>
<body>

</body>
</html>
```


Assignment Operators-

Sr.No.	Operator & Description
1	= (Simple Assignment) Assigns values from the right side operand to the left side operand Ex: $C = A + B$ will assign the value of $A + B$ into C
2	+= (Add and Assignment) It adds the right operand to the left operand and assigns the result to the left operand. Ex: $C += A$ is equivalent to $C = C + A$
3	-= (Subtract and Assignment) It subtracts the right operand from the left operand and assigns the result to the left operand. Ex: $C -= A$ is equivalent to $C = C - A$
4	*= (Multiply and Assignment) It multiplies the right operand with the left operand and assigns the result to the left operand. Ex: $C *= A$ is equivalent to $C = C * A$
5	/= (Divide and Assignment) It divides the left operand with the right operand and assigns the result to the left operand. Ex: $C /= A$ is equivalent to $C = C / A$
6	%= (Modules and Assignment) It takes modulus using two operands and assigns the result to the left operand. Ex: $C \% = A$ is equivalent to $C = C \% A$

Assignment Operators Example-

```
<!DOCTYPE html>
<html lang="en">
<head>
  <meta charset="UTF-8">
  <meta http-equiv="X-UA-Compatible" content="IE=edge">
  <meta name="viewport" content="width=device-width, initial-scale=1.0">
  <title>Document</title>
  <script>

    var a = 33;
    var b = 10;
    var result;
    var newline = "<br />";

    document.write("Value of a => (a = b) => ");
    result = (a = b);
    document.write(result);
    document.write(newline);

    document.write("Value of a => (a += b) => ");
    result = (a += b);
    document.write(result);
    document.write(newline);

    document.write("Value of a => (a -= b) => ");
    result = (a -= b);
    document.write(result);
    document.write(newline);

    document.write("Value of a => (a *= b) => ");
    result = (a *= b);
    document.write(result);
    document.write(newline);

    document.write("Value of a => (a /= b) => ");
    result = (a /= b);
    document.write(result);
    document.write(newline);

    document.write("Value of a => (a %= b) => ");
    result = (a %= b);
    document.write(result);
    document.write(newline);

  </script>
</head>
<body>

</body>
</html>
```

Comparison Operators -

JavaScript supports the following comparison operators –
Assume variable A holds 10 and variable B holds 20, then –

Sr.No.	Operator & Description
1	== (Equal) Checks if the value of two operands are equal or not, if yes, then the condition becomes true. Ex: (A == B) is not true.
2	!= (Not Equal) Checks if the value of two operands are equal or not, if the values are not equal, then the condition becomes true. Ex: (A != B) is true.
3	> (Greater than) Checks if the value of the left operand is greater than the value of the right operand, if yes, then the condition becomes true. Ex: (A > B) is not true.
4	< (Less than) Checks if the value of the left operand is less than the value of the right operand, if yes, then the condition becomes true. Ex: (A < B) is true.
5	>= (Greater than or Equal to) Checks if the value of the left operand is greater than or equal to the value of the right operand, if yes, then the condition becomes true. Ex: (A >= B) is not true.
6	<= (Less than or Equal to) Checks if the value of the left operand is less than or equal to the value of the right operand, if yes, then the condition becomes true. Ex: (A <= B) is true.

Comparison Operators example -

JavaScript supports the following comparison operators –
Assume variable A holds 10 and variable B holds 20, then –

```
<!DOCTYPE html>
<html lang="en">
<head>
  <meta charset="UTF-8">
  <meta http-equiv="X-UA-Compatible" content="IE=edge">
  <meta name="viewport" content="width=device-width, initial-scale=1.0">
  <title>Document</title>
</head>
<body>
  <script>
    var a = 10;
    var result;
    var b = 20;
    var newline = "<br />";

    document.write("(a == b) => ");
    result = (a == b);
    document.write(result);
    document.write(newline);

    document.write("(a < b) => ");
    result = (a < b);
    document.write(result);
    document.write(newline);

    document.write("(a > b) => ");
    result = (a > b);
    document.write(result);
    document.write(newline);

    document.write("(a != b) => ");
    result = (a != b);
    document.write(result);
    document.write(newline);

    document.write("(a >= b) => ");
    result = (a >= b);
    document.write(result);
    document.write(newline);

    document.write("(a <= b) => ");
    result = (a <= b);
    document.write(result);
```

Miscellaneous Operators -

We will discuss two operators here that are quite useful in JavaScript: the **conditional operator (? :)** and the **typeof operator**.

Conditional Operator (? :)

The conditional operator first evaluates an expression for a true or false value and then executes one of the two given statements depending upon the result of the evaluation.

Sr.No.	Operator and Description
1	? : (Conditional) If Condition is true? Then value X : Otherwise value Y

Conditional Operator example -

```
<!DOCTYPE html>
<html lang="en">
<head>
  <meta charset="UTF-8">
  <meta http-equiv="X-UA-Compatible" content="IE=edge">
  <meta name="viewport" content="width=device-width, initial-scale=1.0">
  <title>Document</title>
</head>
<body>
  <script>
    var a = 10;
    var result;

    var b = 20;
    var newline = "<br />";

    document.write ("((a > b) ? 100 : 200) => ");
    result = (a > b) ? 100 : 200;
    document.write(result);
    document.write(newline);

    document.write ("((a < b) ? 100 : 200) => ");
    result = (a < b) ? 100 : 200;
    document.write(result);
    document.write(newline);

  </script>

</body>
</html>
```

Miscellaneous Operators – typeof Operator

- The **typeof** operator is a unary operator that is placed before its single operand, which can be of any type. Its value is a string indicating the data type of the operand.
- The *typeof* operator evaluates to "number", "string", or "boolean" if its operand is a number, string, or boolean value and returns true or false based on the evaluation.

Here is a list of the return values for the **typeof** Operator.

Type	String Returned by typeof
Number	"number"
String	"string"
Boolean	"boolean"
Object	"object"
Function	"function"
Undefined	"undefined"
Null	"object"

Typeof Operator example -

```
<!DOCTYPE html>
<html lang="en">
<head>
  <meta charset="UTF-8">
  <meta http-equiv="X-UA-Compatible" content="IE=edge">
  <meta name="viewport" content="width=device-width, initial-scale=1.0">
  <title>Document</title>
</head>
<body>
  <script>
    var a = 10;
    var b = "String";
    var c=10.2;
    var newline = "<br />";

    result = (typeof b == "string" ? "b is String" : "b is Numeric");
    document.write("Result => ");
    document.write(result);
    document.write(newline);

    result = (typeof a == "string" ? "a is String" : "a is Numeric");
    document.write("Result => ");
    document.write(result);
    document.write(newline);

    result = (typeof c == "string" ? "c is String" : "c is decimal");
    document.write("Result => ");
    document.write(result);
    document.write(newline);
  </script>
</body>
</html>
```


JavaScript – if...else statement

While writing a program, there may be a situation when you need to adopt one out of a given set of paths. In such cases, you need to use conditional statements that allow your program to make correct decisions and perform right actions.

JavaScript supports conditional statements which are used to perform different actions based on different conditions. Here we will discuss the **if..else** statement.

JavaScript supports the following forms of **if..else** statement –

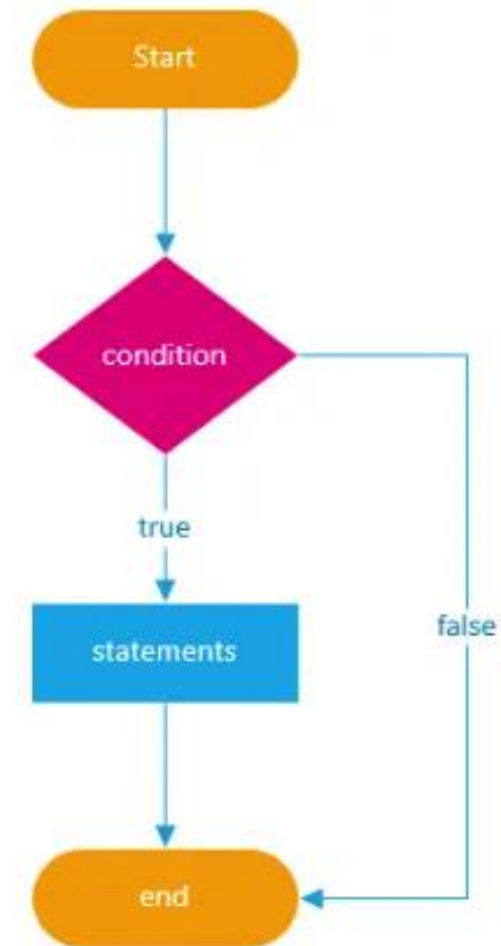
- if statement
- if...else statement
- if...else if... statement.

JavaScript – if statement -

The if statement is the fundamental control statement that allows JavaScript to make decisions and execute statements conditionally.

Syntax-

```
if (expression)
{
    Statement(s) to be executed if expression
    is true
}
```



JavaScript – if statement example-

```
<!DOCTYPE html>
<html lang="en">
<head>
  <meta charset="UTF-8">
  <meta http-equiv="X-UA-Compatible" content="IE=edge">
  <meta name="viewport" content="width=device-width, initial-scale=1.0">
  <title>Document</title>
</head>
<body>
  <script>
    var age = 20;

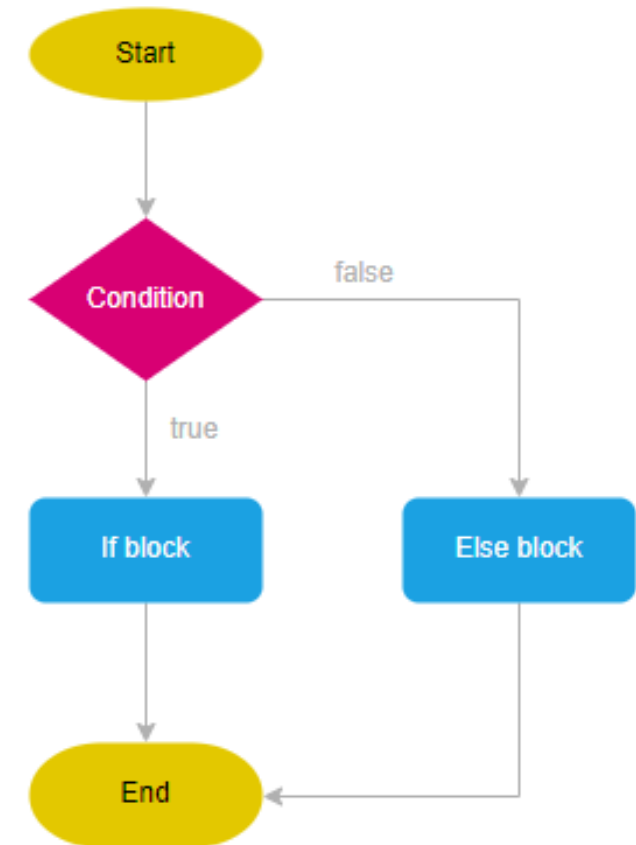
    if( age > 18 ) {
      document.write("<b>Qualifies for driving</b>");
    }
  </script>
</body>
</html>
```

JavaScript – if...else statement flow chart

The '**if...else**' statement is the next form of control statement that allows JavaScript to execute statements in a more controlled way.

Syntax-

```
if (expression) {  
    Statement(s) to be executed if expression is true  
} else {  
    Statement(s) to be executed if expression is false  
}
```



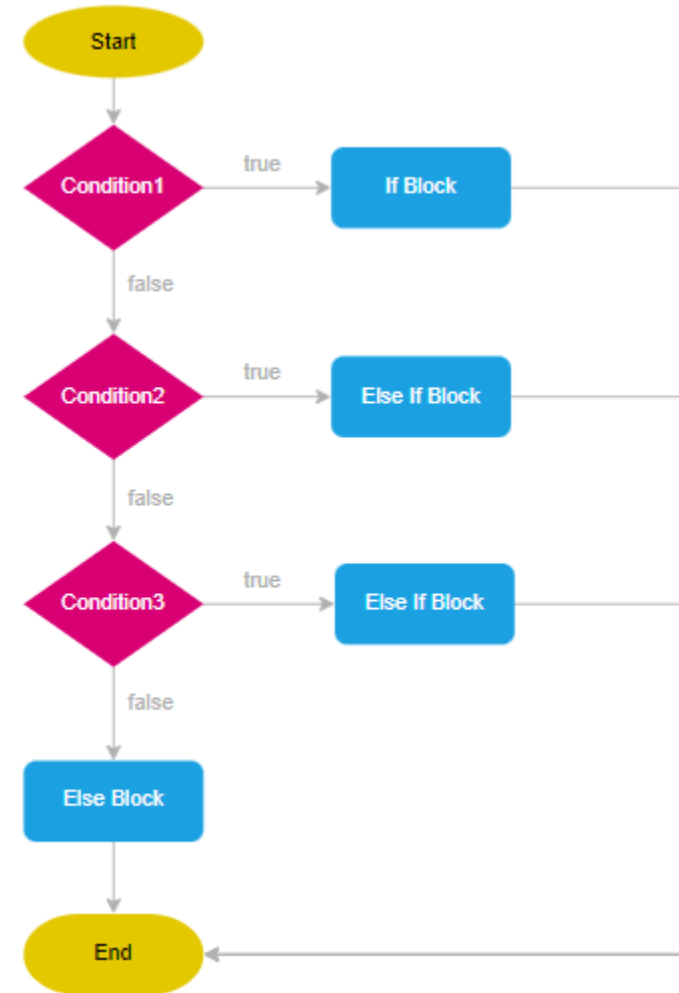
JavaScript – if... else statement example-

```
<!DOCTYPE html>
<html lang="en">
<head>
  <meta charset="UTF-8">
  <meta http-equiv="X-UA-Compatible" content="IE=edge">
  <meta name="viewport" content="width=device-width, initial-scale=1.0">
  <title>Document</title>
</head>
<body>
  <script>
    var age = 15;
    if( age > 18 ) {
      document.write("<b>Qualifies for driving</b>");
    }
    else {
      document.write("<b>Does not qualify for driving</b>");
    }
  </script>
</body>
</html>
```

JavaScript – if...else if.... statement -

The **if...else if...** statement is an advanced form of **if...else** that allows JavaScript to make a correct decision out of several conditions.

```
if (expression 1) {  
    Statement(s) to be executed if expression 1 is true  
}  
else if (expression 2) {  
    Statement(s) to be executed if expression 2 is true  
}  
else if (expression 3) {  
    Statement(s) to be executed if expression 3 is true  
}  
else {  
    Statement(s) to be executed if no expression is true  
}
```



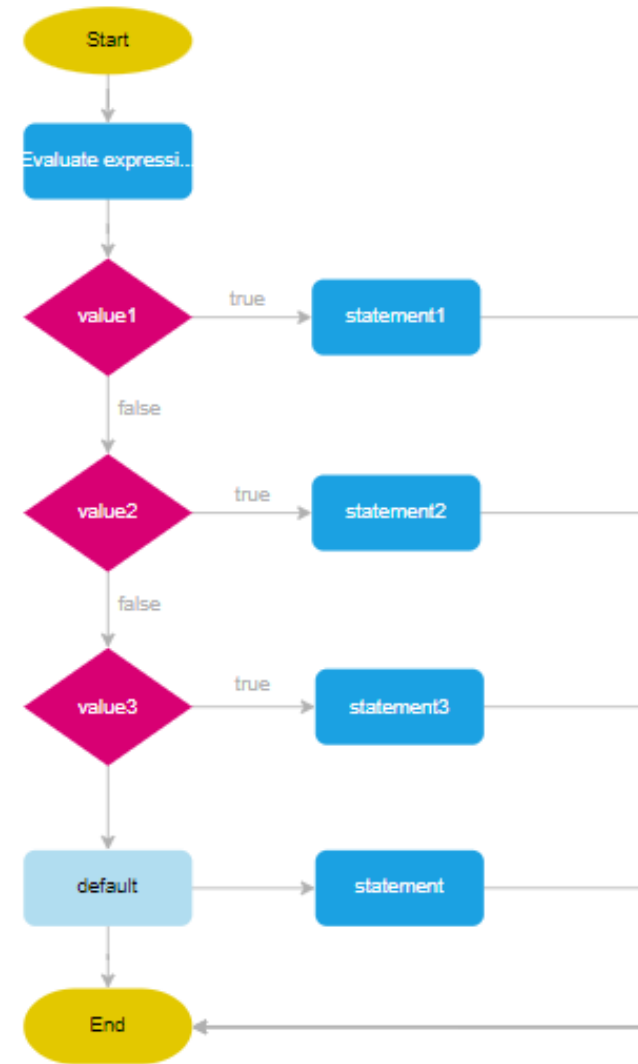
JavaScript – if... else if statement example-

```
<!DOCTYPE html>
<html lang="en">
<head>
  <meta charset="UTF-8">
  <meta http-equiv="X-UA-Compatible" content="IE=edge">
  <meta name="viewport" content="width=device-width, initial-scale=1.0">
  <title>Document</title>
</head>
<body>
  <script>
    var book = "maths";
    if( book == "history" ) {
      document.write("<b>History Book</b>");
    } else if( book == "maths" ) {
      document.write("<b>Maths Book</b>");
    } else if( book == "economics" ) {
      document.write("<b>Economics Book</b>");
    } else {
      document.write("<b>Unknown Book</b>");
    } </script>
  </body>
</html>
```

JavaScript – switch case statement -

You can use multiple **if...else...if** statements, as in the previous slides, to perform a multiway branch. However, this is not always the best solution, especially when all of the branches depend on the value of a single variable. Starting with JavaScript 1.2, you can use a **switch** statement which handles exactly this situation, and it does so more efficiently than repeated **if...else if** statements.

```
switch (expression) {  
  case condition 1: statement(s)  
    break;  
  
  case condition 2: statement(s)  
    break;  
  ...  
  
  case condition n: statement(s)  
    break;  
  
  default: statement(s)  
}
```



JavaScript – switch case example-

```
<!DOCTYPE html>
<html lang="en">
<head>
  <meta charset="UTF-8">
  <meta http-equiv="X-UA-Compatible" content="IE=edge">
  <meta name="viewport" content="width=device-width, initial-scale=1.0">
  <title>Document</title>
</head>
<body>
  <script>
    var grade = 'A';
    document.write("Entering switch block<br />");
    switch (grade) {
      case 'A': document.write("Good job<br />");
        break;

      case 'B': document.write("Pretty good<br />");
        break;

      case 'C': document.write("Passed<br />");
        break;

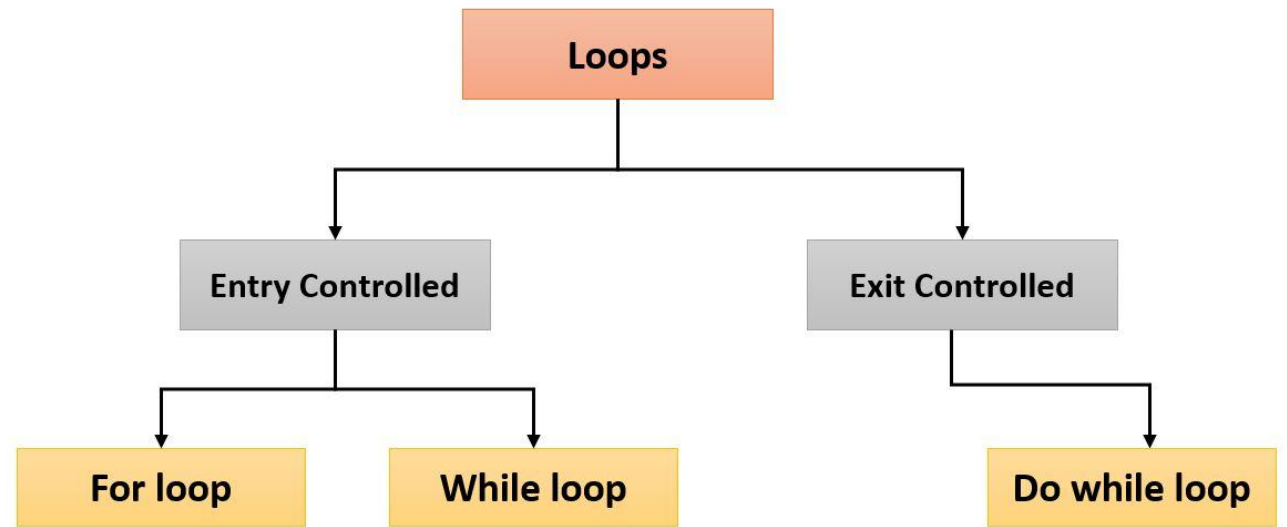
      case 'D': document.write("Not so good<br />");
        break;

      case 'F': document.write("Failed<br />");
        break;

      default: document.write("Unknown grade<br />")
    }
    document.write("Exiting switch block");
  } </script>
</body>
</html>
```

JavaScript – loops -

While writing a program, you may encounter a situation where you need to perform an action over and over again. In such situations, you would need to write loop statements to reduce the number of lines. JavaScript supports all the necessary loops to ease down the pressure of programming.

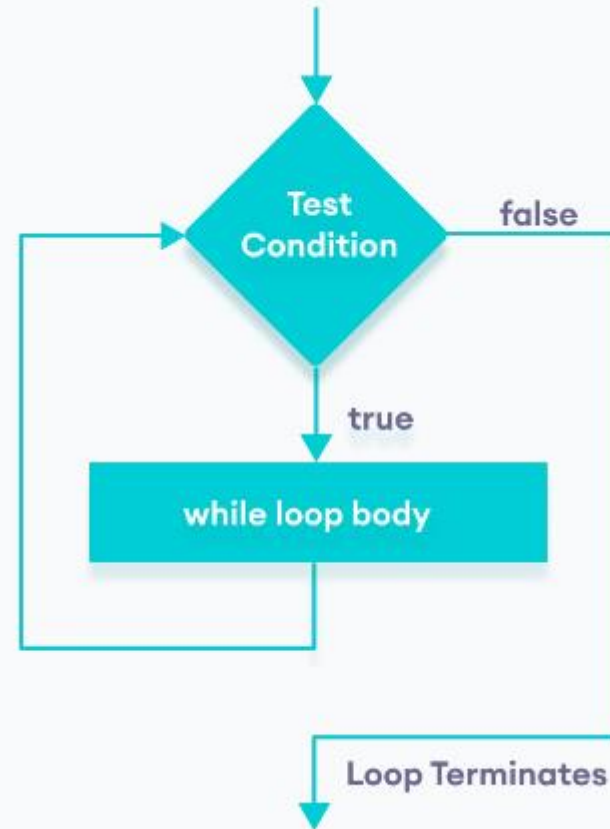


while loop -

The purpose of a **while** loop is to execute a statement or code block repeatedly as long as an **expression** is true. Once the expression becomes **false**, the loop terminates.

Syntax-

```
while (expression) {  
    Statement(s) to be executed if  
    expression is true  
}
```



JavaScript – while loop example-

```
<html>
  <body>
    <script type = "text/javascript">
      var count = 0;
      document.write("Starting Loop ");

      while (count < 10) {
        document.write("Current Count : " + count + "<br />");
        count++;
      }

      document.write("Loop stopped!");

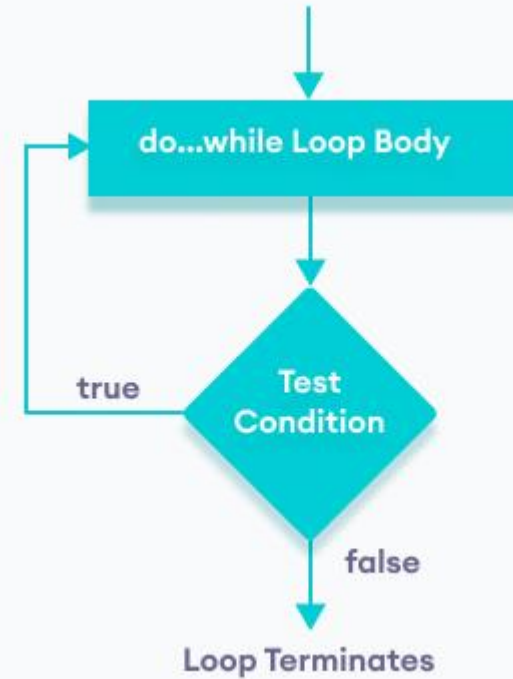
    </script>
  </body>
</html>
```

do while loop -

The **do...while** loop is similar to the **while** loop except that the condition check happens at the end of the loop. This means that the loop will always be executed at least once, even if the condition is **false**

Syntax-

```
do {  
    Statement(s) to be executed;  
} while (expression);
```



JavaScript – do while loop example-

```
<html>
  <body>
    <script type = "text/javascript">
      var count = 0;

      document.write("Starting Loop" + "<br />");
      do {
        document.write("Current Count : " + count + "<br />");
        count++;
      }

      while (count < 5);
      document.write ("Loop stopped!");
    </script>
  </body>
</html>
```

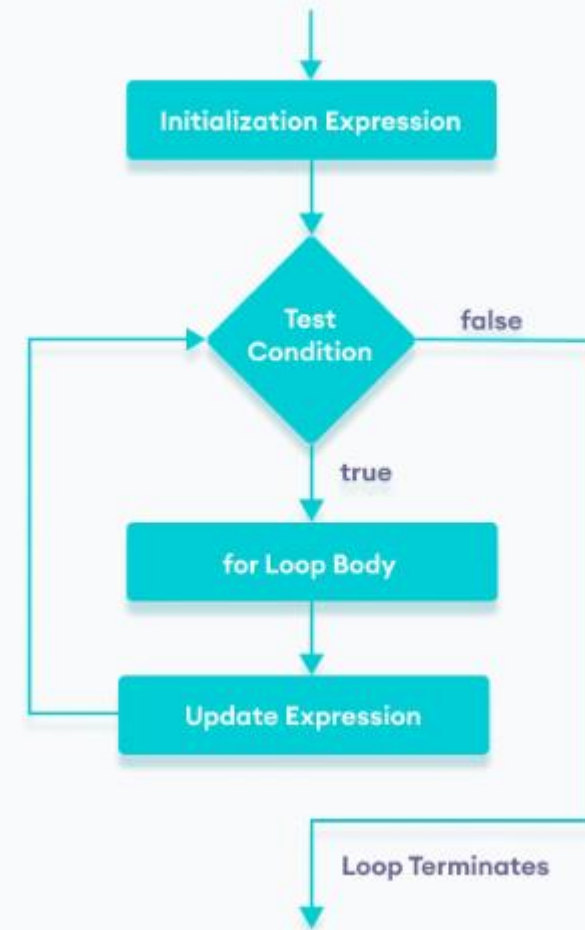
for loop -

The '**for**' loop is the most compact form of looping. It includes the following three important parts –

- The **loop initialization** where we initialize our counter to a starting value. The initialization statement is executed before the loop begins.
- The **test statement** which will test if a given condition is true or not. If the condition is true, then the code given inside the loop will be executed, otherwise the control will come out of the loop.
- The **iteration statement** where you can increase or decrease your counter.

Syntax-

```
for (initialization; test condition; iteration statement)
{
    Statement(s) to be executed if test condition is true
}
```



JavaScript - Loop Control

There may be a situation when you need to come out of a loop without reaching its bottom. There may also be a situation when you want to skip a part of your code block and start the next iteration of the loop.

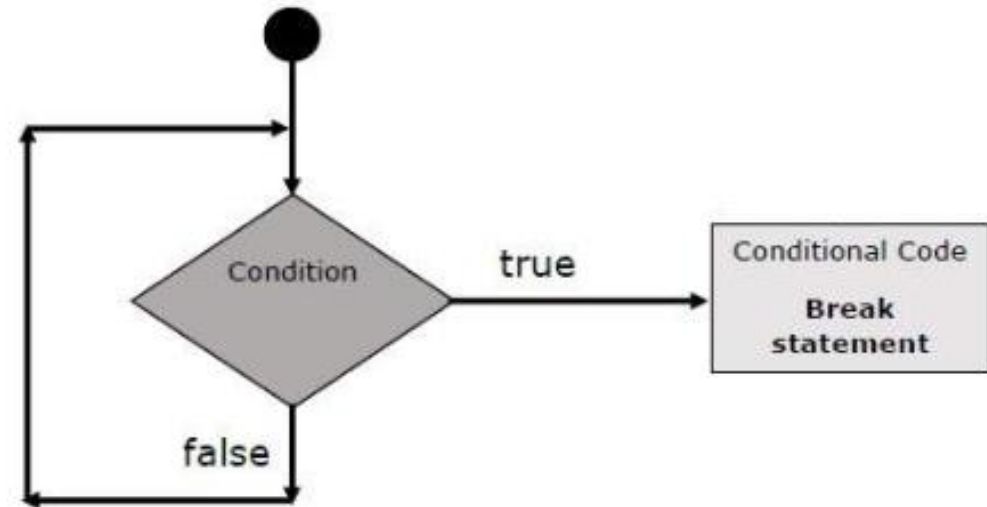
To handle all such situations, JavaScript provides **break** and **continue** statement. These statements are used to immediately come out of any loop or to start the next iteration of any loop respectively.

The break Statement

The **break** statement, which was briefly introduced with the *switch* statement, is used to exit a loop early, breaking out of the enclosing curly braces.

Flow Chart

The flow chart of a break statement would look as follows –



The break Statement - example

The **break** statement, which was briefly introduced with the *switch* statement, is used to exit a loop early, breaking out of the enclosing curly braces.

Output-

Entering the loop

2

3

4

5

Exiting the loop!

```
<!DOCTYPE html>
<html lang="en">
<head>
  <meta charset="UTF-8">
  <meta http-equiv="X-UA-Compatible" content="IE=edge">
  <meta name="viewport" content="width=device-width,
initial-scale=1.0">
  <title>Document</title>
</head>
<body>
  <script type = "text/javascript">
    var x = 1;
    document.write("Entering the loop<br /> ");

    while (x < 20) {
      if (x == 5) {
        break;    // breaks out of loop completely
      }
      x = x + 1;
      document.write( x + "<br />");
    }
    document.write("Exiting the loop!<br /> ");
  </script>
</body>
</html>
```

The continue Statement

The **continue** statement tells the interpreter to immediately start the next iteration of the loop and skip the remaining code block. When a **continue** statement is encountered, the program flow moves to the loop check expression immediately and if the condition remains true, then it starts the next iteration, otherwise the control comes out of the loop

The continue Statement - example

Entering the loop

2

3

4

6

7

8

9

10

Exiting the loop!

```
<!DOCTYPE html>
<html lang="en">
<head>
  <meta charset="UTF-8">
  <meta http-equiv="X-UA-Compatible" content="IE=edge">
  <meta name="viewport" content="width=device-width,
initial-scale=1.0">
  <title>Document</title>
</head>
<body>
  <script type = "text/javascript">
    var x = 1;
    document.write("Entering the loop<br /> ");

    while (x < 10) {
      x = x + 1;

      if (x == 5) {
        continue;    // skip rest of the loop body
      }
      document.write( x + "<br />");
    }
    document.write("Exiting the loop!<br /> ");
  </script>
</body>
</html>
```

The JavaScript - Functions

A function is a group of reusable code which can be called anywhere in your program. This eliminates the need of writing the same code again and again. It helps programmers in writing modular codes. Functions allow a programmer to divide a big program into a number of small and manageable functions.

Steps to implement function-

1. Function definition
2. Calling function

Function definition

Before we use a function, we need to define it. The most common way to define a function in JavaScript is by using the **function** keyword, followed by a unique function name, a list of parameters (that might be empty), and a statement block surrounded by curly braces.

Syntax-

```
<script type = "text/javascript">  
    function functionname(parameter-list) {  
        statements  
    }  
</script>
```

```
<script type = "text/javascript">  
    function Hello()  
    {  
        alert("Hello there");  
    }  
</script>
```

Calling a function -

To invoke a function somewhere later in the script, you would simply call the function-

```
<!DOCTYPE html>
<html lang="en">
<head>
  <meta charset="UTF-8">
  <meta http-equiv="X-UA-Compatible" content="IE=edge">
  <meta name="viewport" content="width=device-width, initial-
scale=1.0">
  <title>Document</title>
  <script>
function Hello()
{
  alert("Hello");
  console.log("function is called");
}

  </script>
</head>
<body>
  <form>
    <input type="button" value="say Hello" onclick="Hello()">
  </form>
</body>
</html>
```

Function Parameters-

Till now, we have seen functions without parameters. But there is a facility to pass different parameters while calling a function. These passed parameters can be captured inside the function and any manipulation can be done over those parameters. A function can take multiple parameters separated by comma.

```
<!DOCTYPE html>
<html lang="en">
<head>
  <meta charset="UTF-8">
  <meta http-equiv="X-UA-Compatible" content="IE=edge">
  <meta name="viewport" content="width=device-width,
initial-scale=1.0">
  <title>Document</title>
  <script>
function Hello(name, age)
{
  document.write("Name:" +name+ " age: "+age);
  console.log("function is called");
}

  </script>
</head>
<body>
  <form>
    <input type="button" value="say Hello"
onclick="Hello('Anil', 25)">
  </form>
</body>
</html>
```


The return statement-

A JavaScript function can have an optional **return** statement. This is required if you want to return a value from a function. This statement should be the last statement in a function.

```
<!DOCTYPE html>
<html lang="en">
<head>
  <meta charset="UTF-8">
  <meta http-equiv="X-UA-Compatible" content="IE=edge">
  <meta name="viewport" content="width=device-width,
initial-scale=1.0">
  <title>Document</title>
  <script>
function concatenate(firstname, lastname)
{
var fullname=firstname + lastname;
return fullname;
}
function Name()
{
  var name=concatenate('Anil',' Giri');
  document.write("Full name is: "+ name);
}
  </script>
</head>
<body>
  <form>
    <input type="button" onclick="Name()"
value="Fullname">
  </form>
</body>
</html>
```

JavaScript Function Literals-

JavaScript 1.2 introduces the concept of **function literals** which is another new way of defining functions. A function literal is an expression that defines an unnamed function.

Syntax-

The syntax for a **function literal** is much like a function statement, except that it is used as an expression rather than a statement and no function name is required.

```
<script type = "text/javascript">  
  var variablename = function(Argument List)  
  {  
    ----- Function Body  
  };  
</script>
```

JavaScript Function Literals example-

```
<!DOCTYPE html>
<html lang="en">
<head>
  <meta charset="UTF-8">
  <meta http-equiv="X-UA-Compatible" content="IE=edge">
  <meta name="viewport" content="width=device-width, initial-scale=1.0">
  <title>Document</title>
  <script type = "text/javascript">
    var func = function(x,y) {
      return x*y
    };
    function myFunction() {
      var result;
      result = func(20,30);
      document.write ( result );
    }
  </script>
</head>

<body>
  <p>Click the following button to call the function</p>
  <form>
    <input type = "button" onclick = "myFunction()" value = "Call
Function">
  </form>
</body>
</html>
```

JavaScript Events-

What is an Event?

Events allow you to write JavaScript code that reacts to certain situations. Examples of events include:

- The user clicks the mouse button
- The Web page loading
- A form field being changed

As of today, there are 2 ways to handle events in JavaScript.

- 1.By using an event handler
- 2.By adding an event listener

JavaScript Events-

Event Handlers

JavaScript provides various kinds of event handlers that get triggered based on specific actions on the HTML elements. Few of the event handlers are:

Event Handler	Description
onclick 	This event handler invokes a JavaScript code when a click action happens on an HTML element. E.g., when we click a button, a link is pushed, a checkbox checks or an image map is selected, it can trigger the onClick event handler.
onload	This event handler invokes a JavaScript code when a window or image finishes loading.
onmouseover	This event handler invokes a JavaScript code when we place the mouse over a specific link or an object.
onmouseout	This event handler invokes a JavaScript code when the mouse leaves a particular link or an object.
onkeypress	This event handler invokes a JavaScript code when the user presses a key.
onkeydown	This event handler invokes a JavaScript code when during the keyboard action, we press the key down.

JavaScript Events-

onclick Event Type

This is the most frequently used event type which occurs when a user clicks the left button of his mouse. You can put your validation, warning etc., against this event type.

```
<html>
  <head>
    <script type = "text/javascript">
      function sayHello() {
        alert("Hello World")
      }

    </script>
  </head>

  <body>
    <p>Click the following button and see result</p>
    <form>
      <input type = "button" onclick = "sayHello()" value = "Say
Hello" />
    </form>
  </body>
</html>
```

JavaScript Events-

onmouseover and onmouseout

These two event types will help you create nice effects with images or even with text as well. The **onmouseover** event triggers when you bring your mouse over any element and the **onmouseout** triggers when you move your mouse out from that element.

Mouse Events

Event	Occurs When
<u>onclick</u>	A user clicks on an element
<u>oncontextmenu</u>	A user right-clicks on an element
<u>ondblclick</u>	A user double-clicks on an element
<u>onmousedown</u>	A mouse button is pressed over an element
<u>onmouseenter</u>	The mouse pointer moves into an element
<u>onmouseleave</u>	The mouse pointer moves out of an element
<u>onmousemove</u>	The mouse pointer moves over an element
<u>onmouseout</u>	The mouse pointer moves out of an element
<u>onmouseover</u>	The mouse pointer moves onto an element
<u>onmouseup</u>	A mouse button is released over an element

JavaScript Events-

onmouseover and onmouseout

These two event types will help you create nice effects with images or even with text as well. The **onmouseover** event triggers when you bring your mouse over any element and the **onmouseout** triggers when you move your mouse out from that element.

```
<!DOCTYPE html>
<html lang="en">
<head>
  <meta charset="UTF-8">
  <meta http-equiv="X-UA-Compatible" content="IE=edge">
  <meta name="viewport" content="width=device-width, initial-scale=1.0">
  <title>Document</title>
  <style>
    div
    {
        width:300px;
        height: 100px;
        background-color: rgb(16, 210, 97);
    }
  </style>
  <script type = "text/javascript">

    function showMessage()
    {
        document.getElementById("name").value="Enter your name";
    }
    function clearText()
    {
        document.getElementById("name").value="";
    }

  </script>
</head>

<body>

  <input type ="text" id="name" onfocus="showMessage()" onmouseover="showMessage()"
onmouseout="clearText()" >
</body>
</html>
```


JavaScript Events-

Focus events

The focus events fire when an element receives or loses focus. These are the two main focus events:

Focus- fires when an element has received focus.

blur - fires when an element has lost focus.

The following elements are focusable:

- The window gains focus when you bring it forward by using Alt+Tab or clicking on it and loses focus when you send it back.
- Links when you use a mouse or a keyboard.
- Form fields like input text when you use a keyboard or a mouse.

```
<html>
<head>
  <script type = "text/javascript">

    function over() {
      document.write ("Mouse Over");
    }
    function out() {
      document.write ("Mouse Out");
    }

  </script>
</head>

<body>
  <p>Bring your mouse inside the division to see the
result:</p>
  <div onmouseover = "over()" onmouseout = "out()">
    <h2> This is inside the division </h2>
  </div>
</body>
</html>
```

JavaScript Events-

Event Listeners

By adding an event listener to an object, we can catch a wide range of events triggered by the user or the browser.

Event Handler	Description
onclick	This event handler invokes a JavaScript code when a click action happens on an HTML element. E.g., when we click a button, a link is pushed, a checkbox checks or an image map is selected, it can trigger the onClick event handler.
onload	This event handler invokes a JavaScript code when a window or image finishes loading.
onmouseover	This event handler invokes a JavaScript code when we place the mouse over a specific link or an object.
onmouseout	This event handler invokes a JavaScript code when the mouse leaves a particular link or an object.
onkeypress	This event handler invokes a JavaScript code when the user presses a key.
onkeydown	This event handler invokes a JavaScript code when during the keyboard action, we press the key down.

JavaScript Events-

HTML Event Attributes :

```
<button onClick="abc()"></button>
```

```

```

JavaScript Events- through external javascript

```
document.getElementById(Id).onclick = functionName;
```

```
document.getElementById("header").onclick = abc;

function abc(){
    document.getElementById("header").style.background = "green";
}
```

JavaScript Events- through external javascript

```
<!DOCTYPE html>
<html lang="en">
<head>
  <meta charset="UTF-8">
  <meta http-equiv="X-UA-Compatible" content="IE=edge">
  <meta name="viewport" content="width=device-width, initial-scale=1.0">
  <title>Document</title>
<style>
#one
{
  width:200px;
  height: 100px;
  border:2px solid black;
  background-color: rgb(42, 216, 16);
}
</style>
</head>

<body>
  <div id="one">
    Click inside div to change the color, default color is green

  </div>
  <script src="event1.js">
  </script>
</body>
</html>
```

```
document.getElementById("one").onclick=fun1;

function fun1()
{
  document.getElementById("one").style.backgroundColor="pink";
}

event1.js
```

JavaScript Events- addEventListener() method-

- When declaring an event listener, we need to provide the event listener with two arguments; “what to listen for” and “what to do when it’s heard.”
- Essentially, we are telling JavaScript to “do *this* after you hear *that*” and attaching that series of commands to an element.

a button
has been
clicked!

a file has
finished
loading!

someone pressed
a keyboard key!

the animation
has started!

event listener



JavaScript Events- addEventListener() method-

Syntax-

```
someElement.addEventListener('What to listen for' , whatToDo)
```

Important: Please make a note that we did *not* include () after the function was passed as the 2nd argument to the event listener. The reason we don't put the () after the function in the event listener declaration is because that would make JavaScript execute the function immediately, which is *not* our goal in adding the event listener.

JavaScript Events- addEventListener() method-

```
document.getElementById(Id).addEventListener("click", functionName);
```

```
document.getElementById(Id).addEventListener("click", function(){
```

Statement

```
});
```

```
document.getElementById("header").addEventListener("click", abc);
```

```
function abc(){
```

```
    document.getElementById("header").style.background = "green";
```

```
}
```


JavaScript Events- addEventListener() method-

```
<!DOCTYPE html>
<html lang="en">
<head>
  <meta charset="UTF-8">
  <meta http-equiv="X-UA-Compatible"
content="IE=edge">
  <meta name="viewport"
content="width=device-width, initial-
scale=1.0">
  <title>Document</title>
</head>
<body>
  <input type="button" id="button1"
value="Click">

  <script src="event1.js">
  </script>
</body>
</html>
```

```
document.getElementById("button1").addE
ventListener("click",fun2);
function fun2()
{
  console.log("Hello Anil I have heard
you...");
}

event1.js
```

JavaScript Events- addEventListener() method-

```
<!DOCTYPE html>
<html lang="en">
<head>
  <meta charset="UTF-8">
  <meta http-equiv="X-UA-Compatible"
content="IE=edge">
  <meta name="viewport"
content="width=device-width, initial-
scale=1.0">
  <title>Document</title>
</head>
<body>
  <input type="button" id="button1"
value="Click">

  <script src="event1.js">
  </script>
</body>
</html>
```

```
document.getElementById("button1").addE
ventListener("click",function()
{
    console.log("Hello Anil I have
heard you...");
});
```

event1.js

JavaScript Events- addEventListener() method- Passing parameter to event function

what if I wanted the *fun2()* function to be dynamic? Instead of the function only having the ability to say "Hello Anil I have heard you...", I want to be able to type in *any* name and then say hello to that.

Although we can pass the argument to the fun2() and get the job done however doing so will run the function without event being heard.

JavaScript Events- addEventListener() method- Passing parameter to event function

So, how do we attach a dynamic function to an event listener? The answer is what we refer to as an anonymous function.

```
<!DOCTYPE html>
<html lang="en">
<head>
  <meta charset="UTF-8">
  <meta http-equiv="X-UA-Compatible"
content="IE=edge">
  <meta name="viewport"
content="width=device-width, initial-
scale=1.0">
  <title>Document</title>
</head>
<body>
  <input type="button" id="button1"
value="Click">

  <script src="event1.js">
  </script>
</body>
</html>
```

```
var username="Rohit";

document.getElementById("button1").addEventL
istener("click",()=>
{
  Name(username);

});
function Name(name)
{
  console.log("Hello "+name+" I have heard
you");
}

event1.js
```

When the callback function for event listeners need arguments passed in, **use anonymous functions.**

JavaScript Events- addEventListener() method-

```
document.getElementById(id).addEventListener("click", functionName);
```

```
document.getElementById(id).addEventListener("click", function(){
```

Statement

```
});
```

```
document.getElementById("header").addEventListener("mouseenter", abc);  
document.getElementById("header").addEventListener("click", function(){  
    document.getElementById("header").style.border = "10px solid red";  
});
```

```
function abc(){  
    document.getElementById("header").style.background = "green";  
}
```

JavaScript Variables

Variables are Containers for Storing Data

JavaScript Variables can be declared in 4 ways:

- Automatically
- Using var
- Using let
- Using const

Note

The var keyword was used in all JavaScript code from 1995 to 2015.

The let and const keywords were added to JavaScript in 2015.

The var keyword should only be used in code written for older browsers.

JavaScript Variables

Example using let

```
let x = 5;  
let y = 6;  
let z = x + y;
```

Example using const

```
const x = 5;  
const y = 6;  
const z = x + y;
```


JavaScript Variables -

When to Use var, let, or const?

1. Always declare variables
2. Always use const if the value should not be changed
3. Always use const if the type should not be changed (Arrays and Objects)
4. Only use let if you can't use const
5. Only use var if you MUST support old browsers.

JavaScript - The Arrays Object

The **Array** object lets you store multiple values in a single variable. It stores a fixed-size sequential collection of elements of the same type. An array is used to store a collection of data, but it is often more useful to think of an array as a collection of variables of the same type.

JavaScript - The Arrays Object

Syntax-

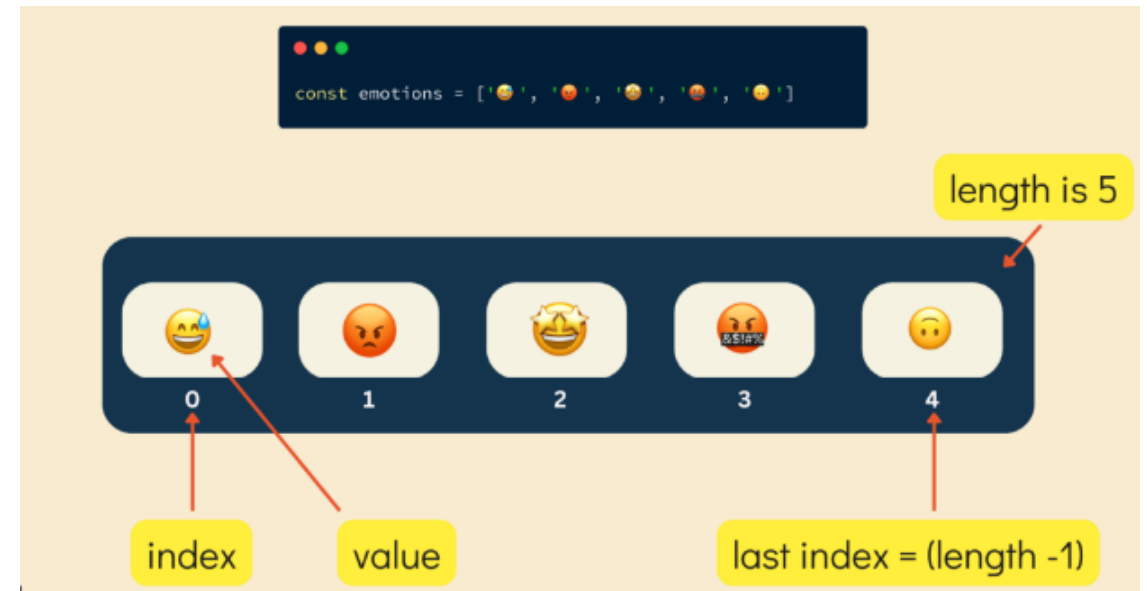
Use the following syntax to create an Array object –

```
var fruits = new Array( "apple", "orange",  
"mango" );
```

The Array parameter is a list of strings or integers. When you specify a single numeric parameter with the Array constructor, you specify the initial length of the array. The maximum length allowed for an array is 4,294,967,295.

You can create array by simply assigning values as follows –

```
var fruits = [ "apple", "orange", "mango" ];
```



fruits[0] is the first element
fruits[1] is the second element
fruits[2] is the third element

JavaScript – Single dimension array

Syntax-

```
let fruits = ["apple", "banana", "orange"];
```

```
console.log(fruits[0]); // Output: "apple"
```

```
console.log(fruits[1]); // Output: "banana"
```

```
console.log(fruits[2]); // Output: "orange"
```

JavaScript - The Arrays examples

```
<!DOCTYPE html>
<html lang="en">
<head>
  <meta charset="UTF-8">
  <meta http-equiv="X-UA-Compatible" content="IE=edge">
  <meta name="viewport" content="width=device-width, initial-scale=1.0">
  <title>Document</title>
  <script>

// let fruits=["Apple","Mango","Grapes","Kiwi","Orange"];
// for(var i=0;i<5;i++)
// {
//   document.write(fruits[i]+" ");
// }

// console.log(fruits[0]);
// document.write(fruits[4]);

let books=[
  "The power of postive thinking",
  1,
  true,
  "Science of getting rich",
  "3AM club"
];

document.write(books); // displaying array values
document.write("<br><br>");

document.write(books[0] +"<br>"); // displaying array values index wise
document.write(books[1] +"<br>");//displaying array values index wise
document.write(books[2] +"<br>");//displaying array values index wise

console.log(books);
document.write("-----Displaying array element using loop-----<br>");
document.write("Number of values stored in array-books is "+books.length+"<br>");
for(var i=0;i<books.length;i++)
{
  document.write(books[i]+"<br>");
}

  </script>
</head>
<body>

</body>
</html>
```

JavaScript - Arrays example using array constructor method

```
<!DOCTYPE html>
<html lang="en">
<head>
  <meta charset="UTF-8">
  <meta http-equiv="X-UA-Compatible" content="IE=edge">
  <meta name="viewport" content="width=device-width, initial-scale=1.0">
  <title>Document</title>
</head>
<body>
  <script>

    let myarr=new Array(5);

    for(let i=0;i<myarr.length;i++)
    {
      myarr[i]=prompt("Please enter the name of employees");
    }

    for(let i=0;i<myarr.length;i++)
    {
      document.write(myarr[i]+"<br>");
    }

  </script>
</body>
</html>
</html>
```

JavaScript – Multidimension array

```
<script>
  var ary = [
    ["Harry",18,"Male","B.Com"],
    ["Sunny",19,"Male","BCA"],
    ["Sarah",18,"Male","BCA"],
    ["Tom",17,"Male","B.A."]
  ];
  document.write(ary);
</script>
```

JavaScript – Printing multidimension array

```
<script>
  var ary = [
    ["Harry",18,"Male","B.Com"],
    ["Sunny",19,"Male","BCA"],
    ["Sarah",18,"Male","BCA"],
    ["Tom",17,"Male","B.A."]
  ];
  for(var a = 0; a < 4; a++){
    document.write(ary[a]);
  }
</script>
```


Javascript array functions-

- `sort()`
- `reverse()`
- `pop()`
- `push()`
- `shift()`
- `unshift()`
- `concat()`
- `slice()`
- `splice()`
- `isArray()`
- `indexOf()`
- `lastIndexOf()`
- `entries()`
- `every()`
- `find()`
- `findIndex()`
- `includes()`
- `some()`
- `forEach()`
- `toString()`
- `valueOf()`

Javascript array functions-sort() and reverse() function

```
<!DOCTYPE html>
<html>
<head>
  <title>JavaScript</title>
  <script>
    var a = ["Sanjay","Aman","Rehman","Karan"];
    document.write(a + "<br><br>");
    a.sort();
    document.write(a + "<br>");
  </script>
</head>
<body>

</body>
</html>
```

```
<script>
  var a = ["Sanjay","Aman","Rehman","Karan"];
  document.write(a + "<br><br>");
  a.sort();
  document.write(a + "<br>");
  a.reverse();
  document.write(a + "<br>");
</script>
```

Javascript array functions-pop() and push() function

```
<!DOCTYPE html>
<html>
<head>
  <title>JavaScript</title>
  <script>
    var a = ["Sanjay","Aman","Rehman","Karan"];
    document.write(a + "<br><br>");
    a.pop();
    document.write(a + "<br><br>");
  </script>
</head>
<body>

</body>
</html>
```

```
<script>
  var a = ["Sanjay","Aman","Rehman","Karan"];
  document.write(a + "<br><br>");
  a.pop();
  document.write(a + "<br><br>");
  a.push("Rahul");
  document.write(a + "<br><br>");
</script>
```

Javascript array functions-shift() and unshift() function

```
<script>
  var a = ["Sanjay","Aman","Rehman","Karan"];
  document.write(a + "<br><br>");
  a.shift();
  document.write(a + "<br><br>");
  a.unshift("Rahul");
  document.write(a + "<br><br>");
</script>
```

Javascript array functions-concat() and join() function

```
<script>
  var a = ["Sanjay", "Aman", "Rehman"];
  var b = ["Rahul", "Karan"];
  var c = a.concat(b);
  document.write(c + "<br><br>");
  var d = c.join(" / ");
  document.write(d);
</script>
```

Javascript array functions-slice() and splice() function

0 1 2 3
var a = ["Sanjay", "Aman", "Rehman", "Karan"];

["Aman", "Rehman"]

slice(start, end)

```
<script>
  var a = ["Sanjay", "Aman", "Rehman",
           "Rahul", "Karan"];

  document.write(a + "<br><br>");

  var b = a.slice(1, 4);
  document.write(b + "<br><br>");
</script>
```

Javascript array functions-slice() and splice() function

```
var a = ["Aman", "Rehman", "Karan"];
      0      1      2
      "Rahul", "Sanjay"
```

`splice(index, howmany, "new value")`
Delete

```
<script>
    var a = ["Sanjay", "Aman", "Rehman", "Rahul"],

    document.write(a + "<br><br>");

    a.splice(2, 0, "Neha", "Karan");
    document.write(a + "<br><br>");
</script>
```

```
<script>
    var a = ["Sanjay","Aman","Rehman","Rahul"],

    document.write(a + "<br><br>");

    a.splice(2,2,"Neha","Karan");
    document.write(a + "<br><br>");
</script>
```


Javascript array functions-indexOf() and lastIndexOf()

0 1 2 3
var a = ["Sanjay", "Aman", "Rehman", "Aman"];
 ▲
 Aman → 1

indexOf("Search item" , Start)

```
<script>
  var a = ["Sanjay", "Aman", "Rehman", "Aman", "Rahul"];
  document.write(a + "<br><br>");

  var b = a.indexOf("Aman");
  document.write(b);
</script>
```


Javascript array functions-indexOf() and lastIndexOf()

Array – lastIndexOf() Function :

0 1 2 3

var a = ["Sanjay","Aman","Rehman","Aman"];

Aman → 3

```
<script>
  var a = ["Sanjay","Aman","Rehman","Aman","Rahul"];
  document.write(a + "<br><br>");

  var b = a.indexOf("Neha", 2);
  document.write(b + "<br><br>");

  var c = a.lastIndexOf("Aman");
  document.write(c + "<br><br>");
</script>
```

Javascript array functions-includes() function

```
var a = ["Sanjay","Aman","Rahul","Karan"];
```

Rahul

includes("Search item")

```
<script>
  var a = ["Sanjay","Aman","Rehman","Rahul"];
  document.write(a + "<br><br>");

  var b = a.includes("Rahul");
  document.write(b + "<br><br>");

</script>
```

Javascript array functions-isArray() function

```
var a = ["Sanjay","Aman","Rehman"];
```

```
var b = 20;
```

```
var c = "Rahul";
```

isArray()

```
<script>
  var a = ["Sanjay","Aman","Rehman"];
  document.write(a + "<br><br>");
  var b = Array.isArray(a);
  document.write(b + "<br><br>");
</script>
```

Javascript array functions-find() and findIndex() function

```
var ages = [10 , 23 , 19 , 20];
```

```
var adultAge = 18;
```

```
ages >= adultAge
```

```
find(Function Name)
```

find() returns the value of the first element in an array that pass a test

```
<script>
  var ages = [10,23,19,20];
  document.write(ages + "<br><br>");

  var b = ages.find(checkAdult);
  document.write(b + "<br><br>");

  function checkAdult(age){
    return age >= 18;
  }
</script>
```

Javascript array functions-find() and findIndex() function

```
var ages = [10 , 23 , 19 , 20];
```

```
var adultAge = 18;
```

```
ages >= adultAge
```

```
findIndex(Function Name)
```

findIndex() returns the index of the first element in an array that pass a test

```
<script>
  var ages = [10,23,19,20];
  document.write(ages + "<br><br>");

  var b = ages.findIndex(checkAdult);
  document.write(b + "<br><br>");

  function checkAdult(age){
    return age >= 18;
  }
</script>
```

Javascript array functions-filter() function

```
var ages = [10 , 23 , 9 , 20];
```

```
var adultAge = 18;
```

```
ages >= adultAge → [23 , 20]
```

```
filter(Function Name)
```

filter() creates an array filled with all array elements that pass a test

```
<script>
  var ages = [10,12,19,20];
  document.write(ages + "<br><br>");

  var b = ages.filter(checkAdult);
  document.write(b + "<br><br>");

  function checkAdult(age){
    return age >= 18;
  }
</script>
```

Javascript array functions-toString() and fill() function

```
var a = ["Sanjay","Aman","Rehman"];
```

Sanjay, Aman, Rehman

toString()

toString() converts an array into String and returns the result

```
<script>
  var a = ["Rahul","Karan","Aman","Neha"];
  a.toString();

  document.write(a);
</script>
```


Javascript array functions-toString() and fill() function

```
var a = ["Aman","Rehman","Karan"];
```

Ram

["Ram","Ram","Ram"]

fill()

```
<script>
  var a = ["Rahul", "Karan", "Aman", "Neha"];
  a.fill("Ram");
  document.write(a);
</script>
```

fill() function fills all the elements in an array with a static value

Javascript array functions-foreach() function

```
var a = ["Sanjay","Aman","Karan"];
```

```
    a.forEach(function(){  
        Statement  
    });
```

The forEach loop in JavaScript allows you to print data without the need for extra variables or testing the length of the array.

```
<script>  
    var a = ["Rahul","Karan","Aman","Neha"];  
  
    a.forEach(function(value){  
        document.write(value + "<br>");  
    });  
</script>
```

```
<script>  
var a=["Rahul","Karan","Aman","Neha"];  
a.forEach(function(value,index)  
{  
    document.write(index+" : "+value+"<br>");  
});  
</script>
```

Javascript array functions-

JavaScript Array length

Returns the number of elements in an array

JavaScript Array reverse()

Returns the array in reverse order

JavaScript Array sort()

Sorts the elements of an array in specific order

JavaScript Array fill()

Returns array by filling elements with given value

Javascript Array join()

Concatenates the array elements to a string

Javascript Array toString()

Returns the string representation of an array

JavaScript Array pop()

Removes and returns the last array element

JavaScript Array shift()

Removes and returns the first array element

JavaScript Array push()

Adds elements to end of array & returns its length

JavaScript - The Strings Object

- The **String** object lets you work with a series of characters; it wraps Javascript's string primitive data type with a number of helper methods.
- As JavaScript automatically converts between string primitives and String objects, you can call any of the helper methods of the String object on a string primitive.

Syntax

Use the following syntax to create a String object –

```
var val = new String(string);
```

JavaScript String - charAt() Method

charAt() is a method that returns the character from the specified index. Characters in a string are indexed from left to right. The index of the first character is 0, and the index of the last character in a string, called **stringName**, is `stringName.length - 1`.

Syntax –

Use the following syntax to find the character at a particular index.
`string.charAt(index);`

Argument Details

index – An integer between 0 and 1 less than the length of the string.

Return Value

Returns the character from the specified index.

JavaScript String - charAt() Method example

```
<html>
  <head>
    <title>JavaScript String charAt() Method</title>
  </head>

  <body>
    <script type = "text/javascript">
      var str = new String( "This is string" );
      document.writeln("str.charAt(0) is:" + str.charAt(0));
      document.writeln("<br />str.charAt(1) is:" + str.charAt(1));
      document.writeln("<br />str.charAt(2) is:" + str.charAt(2));
      document.writeln("<br />str.charAt(3) is:" + str.charAt(3));
      document.writeln("<br />str.charAt(4) is:" + str.charAt(4));
      document.writeln("<br />str.charAt(5) is:" + str.charAt(5));
    </script>
  </body>
</html>
```

JavaScript String - concat() Method

This method adds two or more strings and returns a new single string.

Syntax-

Its syntax is as follows –

```
string.concat(string2, string3[, ..., stringN]);
```

Argument Details

string2...stringN – These are the strings to be concatenated.

Return Value

Returns a single concatenated string.

JavaScript String - concat() Method example

```
<html>
  <head>
    <title>JavaScript String concat() Method</title>
  </head>

  <body>
    <script type = "text/javascript">
      var str1 = new String( "This is string one" );
      var str2 = new String( "This is string two" );
      var str3 = str1.concat( str2 );
      document.write("Concatenated String :" + str3);
    </script>
  </body>
</html>
```

JavaScript String - indexOf() Method

This method returns the index within the calling String object of the first occurrence of the specified value, starting the search at fromIndex or -1 if the value is not found.

Syntax

Use the following syntax to use the indexOf() method.

```
string.indexOf(searchValue[, fromIndex])
```

Argument Details

searchValue – A string representing the value to search for.

fromIndex – The location within the calling string to start the search from. It can be any integer between 0 and the length of the string. The default value is 0.

Return Value

Returns the index of the found occurrence, otherwise -1 if not found.

JavaScript String - indexOf() Method example

```
<html>
  <head>
    <title>JavaScript String indexOf() Method</title>
  </head>

  <body>
    <script type = "text/javascript">
      var str1 = new String( "This is string one" );
      var index1 = str1.indexOf( "string" );
      document.write("indexOf string :" + index1 );

      document.write("<br />");
      var index2 = str1.indexOf( "one" );
      document.write("indexOf one :" + index2 );

    </script>
  </body>
</html>
```

JavaScript String - lastIndexOf() Method

This method returns the index within the calling String object of the last occurrence of the specified value, starting the search at fromIndex or -1 if the value is not found.

Syntax

Its syntax is as follows –

```
string.lastIndexOf(searchValue[, fromIndex])
```

Argument Details

searchValue – A string representing the value to search for.

fromIndex – The location within the calling string to start the search from. It can be any integer between 0 and the length of the string. The default value is 0.

Return Value

Returns the index of the last found occurrence, otherwise -1 if not found.

JavaScript String - lastIndexOf() Method example

```
<html>
  <head>
    <title>JavaScript String lastIndexOf() Method</title>
  </head>

  <body>
    <script type = "text/javascript">
      var str1 = new String( "This is string one and again string" );
      var index = str1.lastIndexOf( "string" );
      document.write("lastIndexOf found String :" + index );

      document.write("<br />");
      var index = str1.lastIndexOf( "one" );
      document.write("lastIndexOf found String :" + index );
    </script>
  </body>
</html>
```

JavaScript String - replace() Method

This method finds a match between a regular expression and a string, and replaces the matched substring with a new substring.

Syntax

Its syntax is as follows –

```
string.replace(searchValue, newValue)
```

Parameters

Parameter	Description
<i>searchValue</i>	Required. The value, or regular expression, to search for.
<i>newValue</i>	Required. The new value (to replace with).

JavaScript String - replace() Method example

```
<html>
  <head>
    <title>JavaScript String lastIndexOf() Method</title>
  </head>

  <body>
    <script type = "text/javascript">

      let text = "Visit Microsoft!";
      let result = text.replace("Microsoft", "Sevenmentor");

      document.write(result );

    </script>
  </body>
</html>
```

JavaScript String - search() Method

The search() method searches a string for a string (or a regular expression) and returns the position of the match:

Syntax

Its syntax is as follows –

```
string.search(regex);
```

JavaScript String - search() Method example

```
<html>
  <head>
    <title>JavaScript String lastIndexOf() Method</title>
  </head>

  <body>
    <script type = "text/javascript">

      let text = "Please locate where 'locate' occurs!";
      let index = text.search("locate");
      document.write(index );

    </script>
  </body>
</html>
```

JavaScript - The Strings Object common methods

```
<html>
  <head>
    <title>JavaScript String charAt() Method</title>
  </head>

  <body>
    <script type = "text/javascript">
const text1 = 'hello';
const text2 = 'world';
const text3 = '    JavaScript    ';

    // concatenating two strings
const result1 = text1.concat(' ', text2);
console.log(result1); // "hello world"

    // converting the text to uppercase
const result2 = text1.toUpperCase();
console.log(result2); // HELLO

    // removing whitespace from the string
const result3 = text3.trim();
console.log(result3); // JavaScript

    // slicing the string
const result5= text1.slice(1, 3);
console.log(result5); // "el"
    </script>
  </body>
</html>
```


JavaScript DOM

A Document object represents the HTML document that is displayed in that window

The Document object has various properties that refer to other objects which allow access to and modification of document content.

The way a document content is accessed and modified is called the **Document Object Model**, or **DOM**. The Objects are organized in a hierarchy. This hierarchical structure applies to the organization of objects in a Web document.

- Window object** – Top of the hierarchy. It is the outmost element of the object hierarchy.
- Document object** – Each HTML document that gets loaded into a window becomes a document object. The document contains the contents of the page.
- Form object** – Everything enclosed in the <form>...</form> tags sets the form object.
- Form control elements** – The form object contains all the elements defined for that object such as text fields, buttons, radio buttons, and checkboxes.

DOM

DOM is such a technology, with the help of which we can completely control any html document with javascript.

With the help of DOM, we can access and change any tag, id, class or any attribute, even we can change any CSS style from DOM.

JavaScript DOM

What is the DOM?

The DOM is a W3C (World Wide Web Consortium) standard.

The DOM defines a standard for accessing documents:

"The W3C Document Object Model (DOM) is a platform and language-neutral interface that allows programs and scripts to dynamically access and update the content, structure, and style of a document."

The W3C DOM standard is separated into 3 different parts:

- **Core DOM** - standard model for all document types
- **XML DOM** - standard model for XML documents
- **HTML DOM** - standard model for HTML documents

What is HTML DOM?

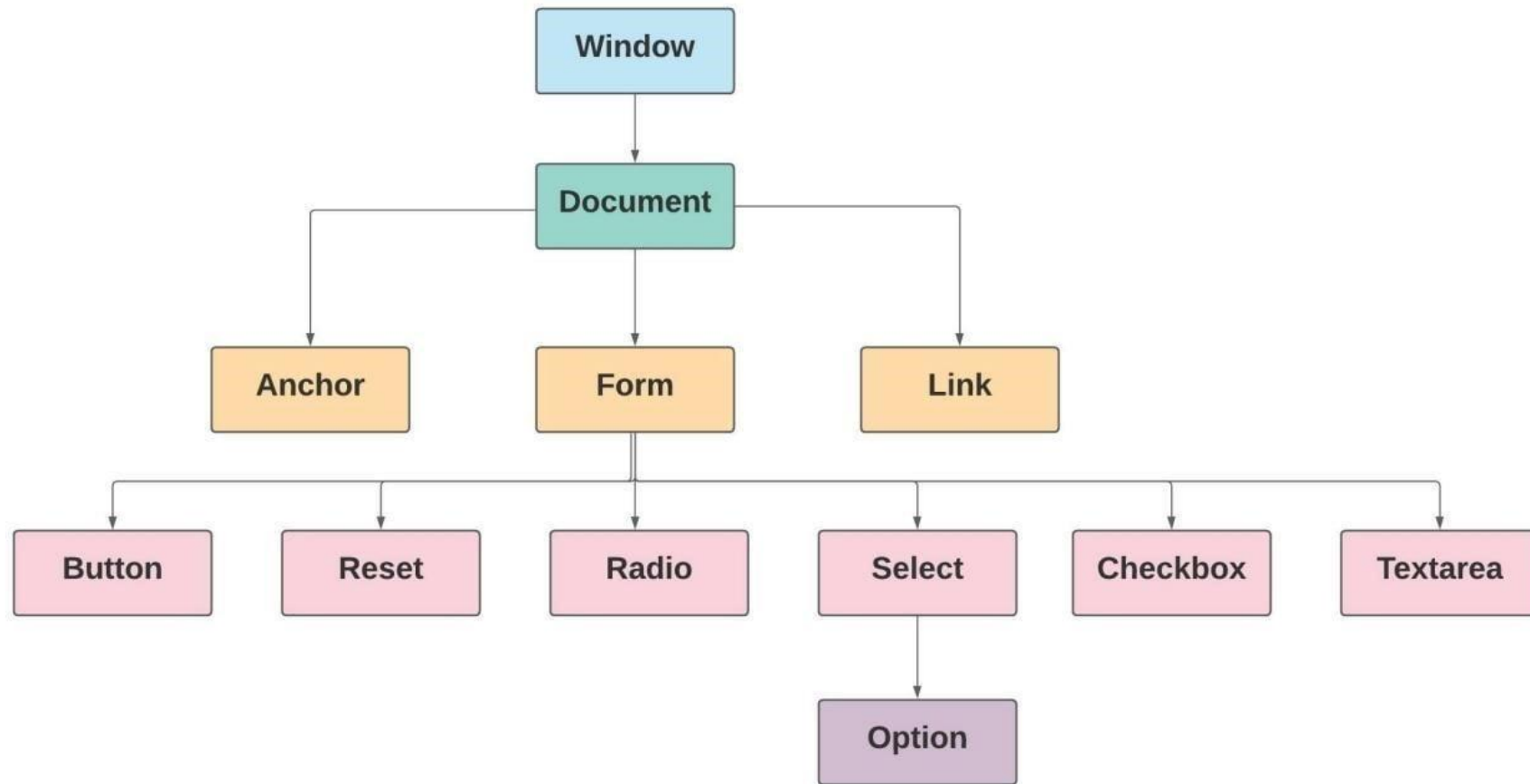
The HTML DOM is a standard **object** model and **programming interface** for HTML. It defines:

- The HTML elements as **objects**
- The **properties** of all HTML elements
- The **methods** to access all HTML elements
- The **events** for all HTML elements

In other words: **The HTML DOM is a standard for how to get, change, add, or delete HTML elements.**

JavaScript DOM - hierarchy

Representation of HTML DOM



Finding HTML Elements

Method	Description
<code>document.getElementById(<i>id</i>)</code>	Find an element by element id
<code>document.getElementsByTagName(<i>name</i>)</code>	Find elements by tag name
<code>document.getElementsByClassName(<i>name</i>)</code>	Find elements by class name

JavaScript DOM

Methods

`getElementById()`

`getElementsByClassName()`

`getElementsByTagName()`

`querySelector()`

`querySelectorAll()`

Method	Description
<code>write("string")</code>	writes the given string on the document.
<code>writeln("string")</code>	writes the given string on the document with newline character at the end.
<code>getElementById()</code>	returns the element having the given id value.
<code>getElementsByName()</code>	returns all the elements having the given name value.
<code>getElementsByTagName()</code>	returns all the elements having the given tag name.
<code>getElementsByClassName()</code>	returns all the elements having the given class name.

Changing HTML Elements

Property	Description
<code>element.innerHTML = new html content</code>	Change the inner HTML of an element
<code>element.attribute = new value</code>	Change the attribute value of an HTML element
<code>element.style.property = new style</code>	Change the style of an HTML element

JavaScript DOM- Accessing field value

```
<script type="text/javascript">  
function printvalue(){  
var name=document.form1.name.value;  
alert("Welcome: "+name);  
}  
</script>
```

```
<form name="form1">  
Enter Name:<input type="text" name="name"/>  
<input type="button" onclick="printvalue()" value="print name  
"/>  
</form>
```


JavaScript DOM- document.getElementById() method

The **document.getElementById()** method returns the element of specified id.

```
<script type="text/javascript">
function getcube(){
var number=document.getElementById("number").value;
alert(number*number*number);
}
</script>
<form>
Enter No:<input type="text" id="number" name="number"/><
br/>
<input type="button" value="cube" onclick="getcube()"/>
</form>
```

JavaScript DOM- document.getElementById() method

```
<!DOCTYPE html>
<html lang="en">
<head>
  <meta charset="UTF-8">
  <meta http-equiv="X-UA-Compatible" content="IE=edge">
  <meta name="viewport" content="width=device-width, initial-scale=1.0">
  <title>Javascript DOM</title>
  <link rel="stylesheet" href="dom1.css">
</head>
  <header id="header">
<h2 id="header-title">Anil Giri</h2>
<p id="tagline">Corporate trainer</p>
  </header>
  <div id="content">
    <h4>Courses</h4>
    <ul id="items">
      <li class="list-item">Java Full stack</li>
      <li class="list-item">.net Full stack</li>
      <li class="list-item">Database</li>
      <li class="list-item">Quality Control</li>
    </ul>
  <form action="">
    <input type="text">
    <button>Submit</button>
  </form>
  </div>
<script src="myscript.js"></script>
<body>

</body>
</html>
```

JavaScript DOM- document.getElementById() method

```
var headerTitle=document.getElementById('header-title');  
console.log(headerTitle);  
headerTitle.textContent='Computer Courses';
```

“myscript.js”

```
*  
{  
    margin: 0;  
    padding: 0;  
}  
body  
{  
    font-family: Arial, Helvetica, sans-serif;  
}  
header  
{  
    background-color: green;  
    color: white;  
    padding: 15px 0px 20px 40px;  
}  
  
h2#header-title  
{  
    padding: 10px 0px;  
}  
  
div#content  
{  
    padding: 15px 0px 20px 45px;  
}  
div#content>h4  
{  
    border-bottom: 2px solid green;  
    padding: 0px 20px 10px 5px;  
    display: inline-block;  
    width: 50%;  
}  
  
li.list-item  
{  
    border-bottom: 1px solid gray;  
    padding: 10px 0px;  
}  
form  
{  
    padding-top: 20px;  
}  
form>input,button  
{  
    padding: 5px;  
}
```

dom1.css

JavaScript DOM- document.getElementsByName() method

The **document.getElementsByName()** method returns all the element of specified name.

```
<script type="text/javascript">
function totalelements()
{
var allgenders=document.getElementsByName("gender");
alert("Total Genders:"+allgenders.length);
}
</script>
<form>
Male:<input type="radio" name="gender" value="male">
Female:<input type="radio" name="gender" value="female">

<input type="button" onclick="totalelements()" value="Total G
enders">
</form>
```

JavaScript DOM- document.getElementsByTagName() method

The **document.getElementsByTagName()** method returns all the element of specified tag name.

```
<script type="text/javascript">
function countpara(){
var totalpara=document.getElementsByTagName("p");
alert("total p tags are: "+totalpara.length);

}
</script>
<p>This is a pragraph</p>
<p>Here we are going to count total number of paragraphs by
getElementByTagName() method.</p>
<p>Let's see the simple example</p>
<button onclick="countpara()">count paragraph</button>
```

JavaScript DOM- document.getElementsByClassName() method

The **document.getElementsByClassName()** method returns all the element of specified class name.

```
var items=document.getElementsByClassName('list-item');
console.log(items);
console.log(items[1]);
items[0].textContent='C#';
items[1].textContent='ASP.net';
items[2].textContent='Web Services';
items[3].textContent='Rest API';
```

JavaScript DOM- Assignment

Anil Giri

Corporate trainer

Courses

- C#
- ASP.net
- **Web Services**
- Rest API

- Change the font color of 3rd item to blue
- Change the background color of 2nd item to yellow
- Replicate the same by using `getElementsByTagName()`

Changing HTML Elements

Property	Description
<code>element.innerHTML = new html content</code>	Change the inner HTML of an element
<code>element.attribute = new value</code>	Change the attribute value of an HTML element
<code>element.style.property = new style</code>	Change the style of an HTML element

Javascript - innerHTML

- The **innerHTML** property can be used to write the dynamic html on the html document.
- It is used mostly in the web pages to generate the dynamic html such as registration form, comment form, links etc.
- The easiest way to get the content of an element is by using the innerHTML property.
- The innerHTML property is useful for getting or replacing the content of HTML elements

```
<script type="text/javascript" >
function showcommentform() {

document.getElementById('mylocation').innerHTML="Writing
inside div";
}
</script>
<form name="myForm">
<input type="button" value="comment" onclick="showcommentform()">
<div id="mylocation"></div>
</form>
```

Changing HTML Elements

```
var headerTitle=document.getElementById('header-title');  
console.log(headerTitle);  
headerTitle.innerText='Computer Courses';
```

```
console.log(headerTitle);  
headerTitle.innerHTML='<h1>Hello</h1>'
```

```
headerTitle.style.borderBottom='5px solid blue';
```

myscript.js

Assignment-

Computer Courses

Corporate trainer

Courses

- Java Full stack
- .net Full stack
- Database
- Quality Control

Submit

Apply the border below header

JavaScript DOM- document.querySelector() method

The **document.querySelector()** is a method used for searching and returning the very first element within the document that matches the given selector.

The queryselector in javascript only returns the element that matches with one of the specified CSS selectors, or a group of selectors.

Note - querySelector() – catches first value that is where querySelectorAll() method comes in picture

querySelectorAll()- works on any item, any index number

```
var headline = document.querySelector('#tagline');  
headline.style.color = 'yellow';
```

```
var headline = document.querySelector('#tagline');  
headline.style.color = 'yellow';  
  
var items = document.querySelector('.list-item');  
items.style.color = 'red';|
```

JavaScript DOM- changing any element without using querySelectorAll() – using pseudoclass

```
var items = document.querySelector('.list-  
item:last-child');  
items.style.color = 'red';
```

Adding and deleting Elements

Method	Description
<code>document.createElement(<i>element</i>)</code>	Create an HTML element
<code>document.removeChild(<i>element</i>)</code>	Remove an HTML element
<code>document.appendChild(<i>element</i>)</code>	Add an HTML element
<code>document.replaceChild(<i>new</i>, <i>old</i>)</code>	Replace an HTML element
<code>document.write(<i>text</i>)</code>	Write into the HTML output stream

Adding Elements

```
var headerTitle=document.getElementById('header-title');
// console.log(headerTitle);
// headerTitle.innerText='IT technology trainer';

// console.log(headerTitle);
// headerTitle.innerHTML='<h1>Computer Courses</h1>';

// headerTitle.style.color='black';

//headerTitle.style.borderBottom='5px solid blue';

// var items=document.getElementsByClassName('list-item');
// console.log(items);
// console.log(items[1]);
// items[0].textContent='C#';
// items[1].textContent='ASP.net';
// items[2].textContent='Web Services';
// items[3].textContent='Rest API';
// items[2].style.fontWeight='bold';

// var mylist=document.getElementsByClassName('list-item');
// console.log(mylist);

// console.log(mylist[3]);
// mylist[0].innerText='Core java';
// mylist[1].innerText='JDBC';
// mylist[2].innerText='Spring';
// mylist[3].innerText='Rest API';
// mylist[2].style.fontWeight='bold';

// var myheading=document.querySelectorAll('.list-item');
// console.log(myheading);
// for(var i=0;i<myheading.length;i++)
// {
// myheading[i].style.backgroundColor='pink';
// }

// creating element, creating div
var newDiv=document.createElement('div');

// adding class
newDiv.className='box';

//adding id
newDiv.id='textBox';

//creating textnode
var newDivText=document.createTextNode('Anil sir teachers all these and more technologies');

//add text to div
newDiv.appendChild(newDivText);

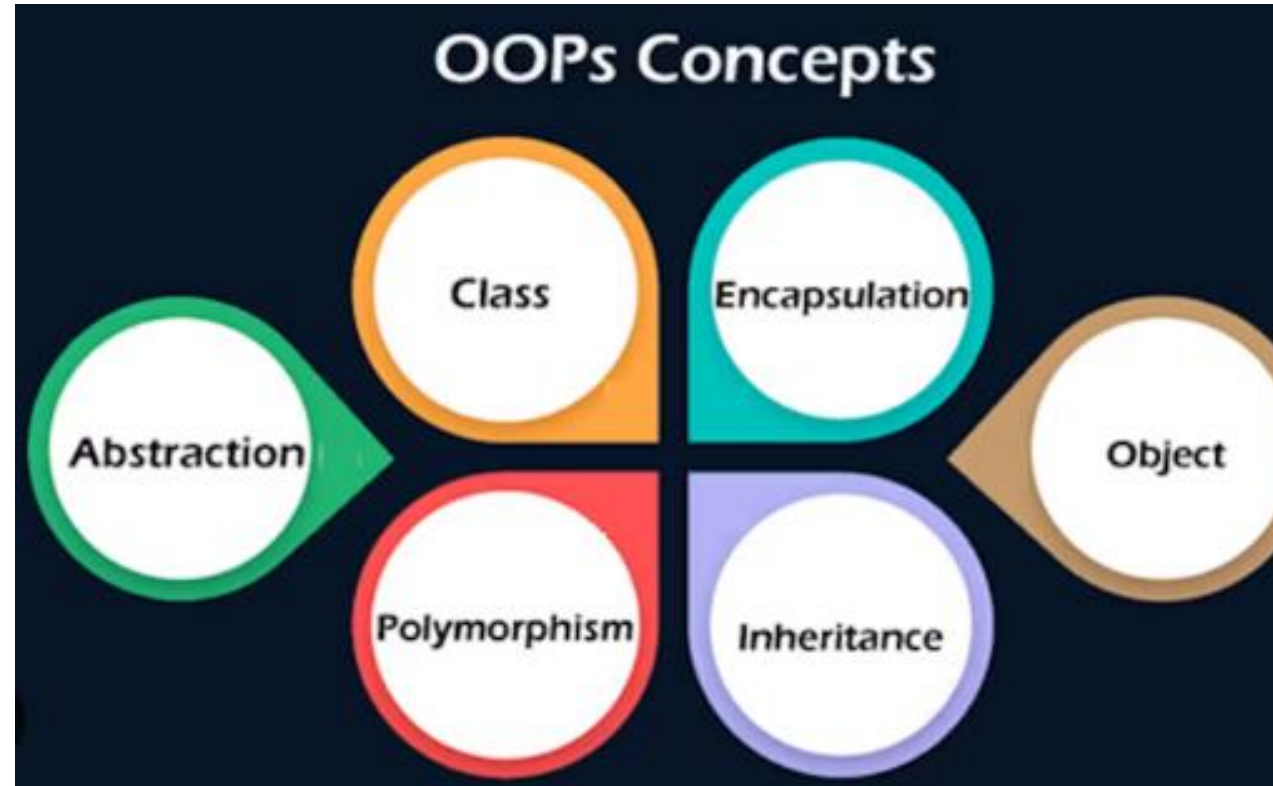
var container=document.querySelector('#content');
var form=document.querySelector('form');
newDiv.style.color='blue';
newDiv.style.backgroundColor='skyblue';
newDiv.style.border='2px solid black';
newDiv.style.padding='10px';
newDiv.style.margin='30px 90px 20px 0px';
// adding div
container.insertBefore(newDiv,form);

//console.log(newDiv);
```

Advanced JavaScript

Oops introduction

Object Oriented Concepts



JS What is OOP ?

Object Oriented Programming

ES6 Version



Class

Coding methodology / style / pattern

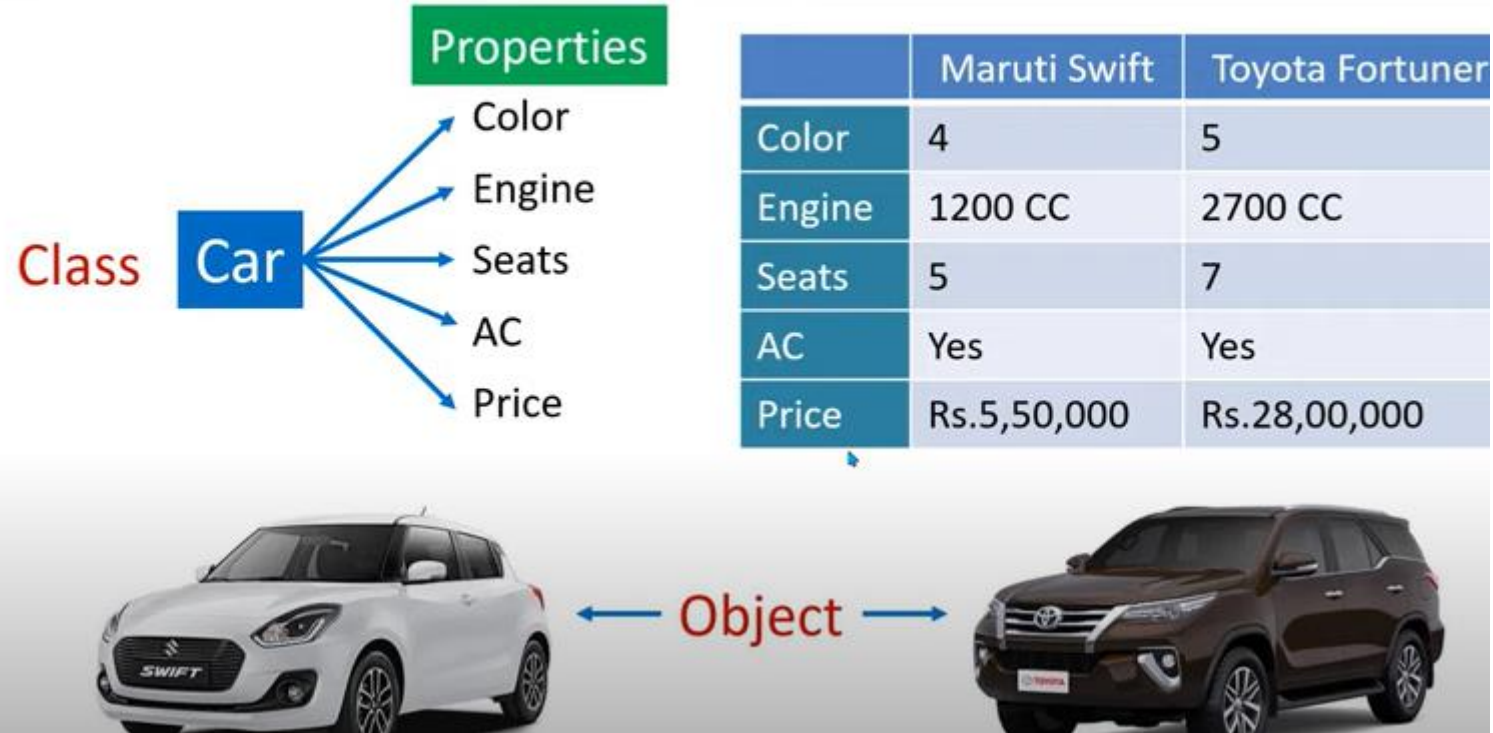
Object

- Code more Modular and Reusable
- Well Organized Code
- Easier to debug
- Best for medium to large website projects

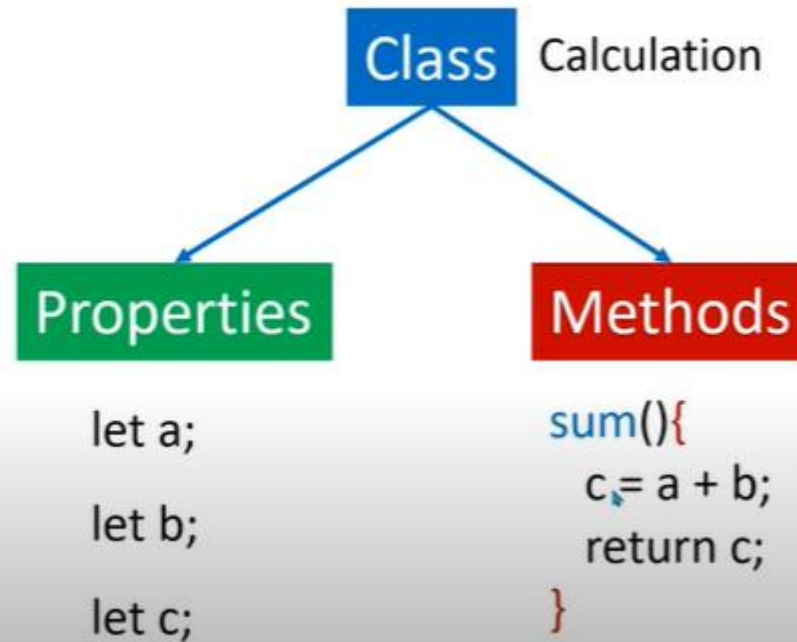
JS What is Class & Object ?



JS What is Class & Object ?



JS What is Properties & Methods ?



JS Class & Object

```
class hello{  
  message(){  
    console.log("Hello Everyone");  
  }  
}
```

```
let a = new hello();  
a.message();
```

JS Class & Object

```
<!DOCTYPE html>
<html lang="en">
<head>
  <meta charset="UTF-8">
  <meta http-equiv="X-UA-Compatible" content="IE=edge">
  <meta name="viewport" content="width=device-width, initial-
scale=1.0">
  <title>Document</title>
  <script>

    class Myclass
    {
      message()
      {
        console.log("hello welcome to oops programming");
      }
    }

    let first=new Myclass();
    first.message();

  </script>
</head>
<body>

</body>
</html>
```

JS Type of Methods :

Constructor

```
constructor(){  
  console.log("hello");  
}
```

Prototype

```
message(){  
  console.log("hello")  
}
```

Static

```
static name(){  
  console.log("hello")  
}
```


Constructor

```
<!DOCTYPE html>
<html lang="en">
<head>
  <meta charset="UTF-8">
  <meta http-equiv="X-UA-Compatible" content="IE=edge">
  <meta name="viewport" content="width=device-width, initial-scale=1.0">
  <title>Document</title>
  <script>

    class MyClass
    {

      constructor(name)
      {
        console.log(name)
      }
      message()
      {
        console.log("hello welcome to oops programming");
      }
    }

    let first=new MyClass("Anil");
    first.message();

  </script>
</head>
<body>

</body>
</html>
```

Static

```
<!DOCTYPE html>
<html lang="en">
<head>
  <meta charset="UTF-8">
  <meta http-equiv="X-UA-Compatible" content="IE=edge">
  <meta name="viewport" content="width=device-width, initial-scale=1.0">
  <title>Document</title>
  <script>

    class Myclass
    {

      constructor(name)
      {
        console.log(name)
        document.write("I am constructor"+"<br>");
      }
      message()
      {
        console.log("hello welcome to oops programming");
        document.write("I am prototype method"+"<br>");
      }
      static mymethod()
      {
        console.log("I am static method");
        document.write("I am static method");
      }
    }

    let first=new Myclass("Anil");
    first.message();
    Myclass.mymethod();

  </script>
</head>
<body>
</body>
</html>
```

Static

```
<script>
  class Example
  {
    static staticVariable = 'Sevenmentor';
    //static variable defined

    static staticMethod()
    {
      return 'static method has been called.';
    }
  }
  // static variable called

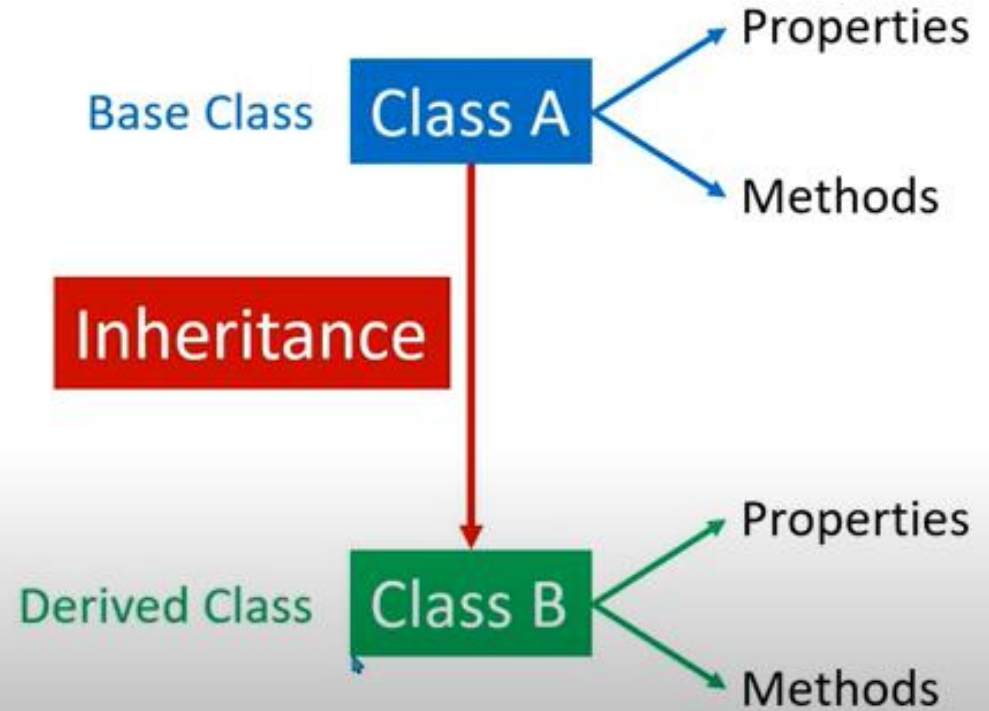
  console.log(Example.staticVariable);
  // static method called
  console.log(Example.staticMethod());
</script>
```

Advance JS

Inheritance

JS What is Inheritance ?

Inheritance in JavaScript is defined as the ability of a class to derive properties and characteristics from another class while having its own properties as well.



```
<script>

  class employee{
    constructor(){
      console.log("Constructor : Employee");
    }
  }

  class manager extends employee{
    
  }

  let a = new manager();

</script>
```

JS Inheritance in JavaScript

```
class fruits{  
    // Properties & Methods  
}  
  
class vegetables extends fruits{  
    // Properties & Methods  
}  
  
let f = new fruits();  
  
let v = new vegetables();
```

```
<script>  
  
    class employee{  
        constructor(){  
            console.log("Constructor : Employee");  
        }  
    }  
  
    class manager extends employee{  
  
    }  
  
    let a = new manager();  
  
</script>
```

Using Constructor -

```
<!DOCTYPE html>
<html lang="en">
<head>
  <meta charset="UTF-8">
  <meta http-equiv="X-UA-Compatible" content="IE=edge">
  <meta name="viewport" content="width=device-width, initial-
scale=1.0">
  <title>Document</title>
  <script>
class Employee
{
  constructor(name)
  {
    console.log("Constructor: Employee "+name);
  }
}
class Manager extends Employee
{
  constructor(myname)
  {
    super();
    console.log("Constructor: Manager "+myname);
  }
}
let obj=new Manager("Anil");

</script>
</head>
<body>

</body>
</html>
```

```
<!DOCTYPE html>
<html lang="en">
<head>
  <meta charset="UTF-8">
  <meta http-equiv="X-UA-Compatible" content="IE=edge">
  <meta name="viewport" content="width=device-width, initial-
scale=1.0">
  <title>Document</title>
  <script>
class Employee
{
  constructor(name)
  {
    console.log("Constructor: Employee"+name);
  }
}
class Manager extends Employee
{
  constructor(myname)
  {
    super();
    console.log("Constructor: Manager"+myname);
  }
}
let obj=new Manager("Anil");

</script>
</head>
<body>

</body>
</html>
```


Assignments -

- Write a JavaScript program that creates a class called "BankAccount" with properties for account number and balance. Include methods to deposit and withdraw money from the account. Create some instances of the "BankAccount" class, deposit some money, and withdraw a portion of it
- Write a JavaScript program that creates a class called 'Shape' with a method to calculate the area. Create two subclasses, 'Circle' and 'Triangle', that inherit from the 'Shape' class and override the area calculation method. Create an instance of the 'Circle' class and calculate its area. Similarly, do the same for the 'Triangle' class.
- Write a JavaScript program that creates a class called 'Employee' with properties for name and salary. Include a method to calculate annual salary. Create a subclass called 'Manager' that inherits from the 'Employee' class and adds an additional property for department. Override the annual salary calculation method to include bonuses for managers. Create two instances of the 'Manager' class and calculate their annual salary

Topics to Discuss

1 Asynchronous Javascript ?

2 Synchronous Vs
Asynchronous

3 Callbacks

4 Promises

5 Async / Await

6 Conclusion



Steps To make Ice Cream

- #1 Place Order
- #2 Cut The Fruit
- #3 Add water and ice
- #4 Start the machine
- #5 Select Container
- #6 Select Toppings
- #7 Serve Ice Cream



Time(seconds)

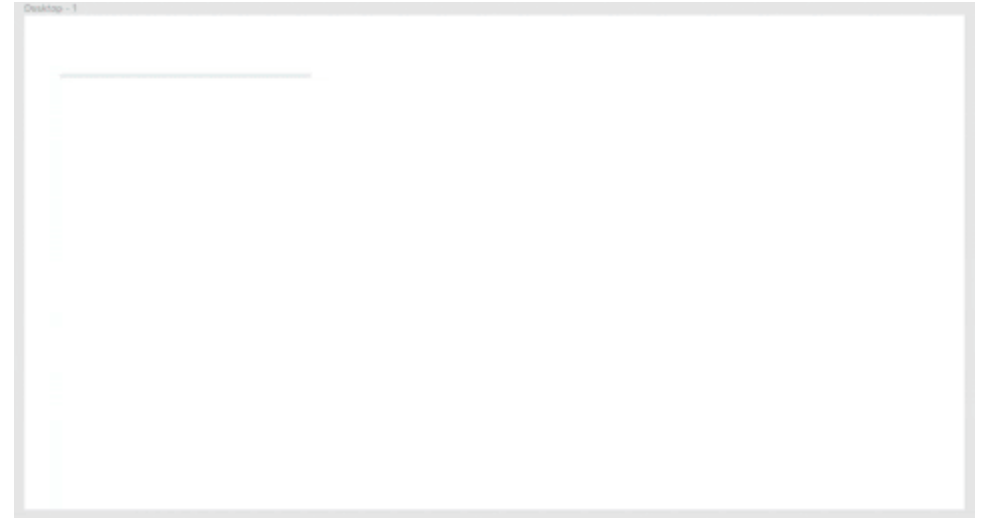
- #1 Place Order → 2
- #2 Cut The Fruit → 2
- #3 Add water and ice → 1
- #4 Start the machine → 1
- #5 Select Container → 2
- #6 Select Toppings → 3
- #7 Serve Ice Cream → 2

Synchronous System -

In a synchronous system, tasks are completed one after another.

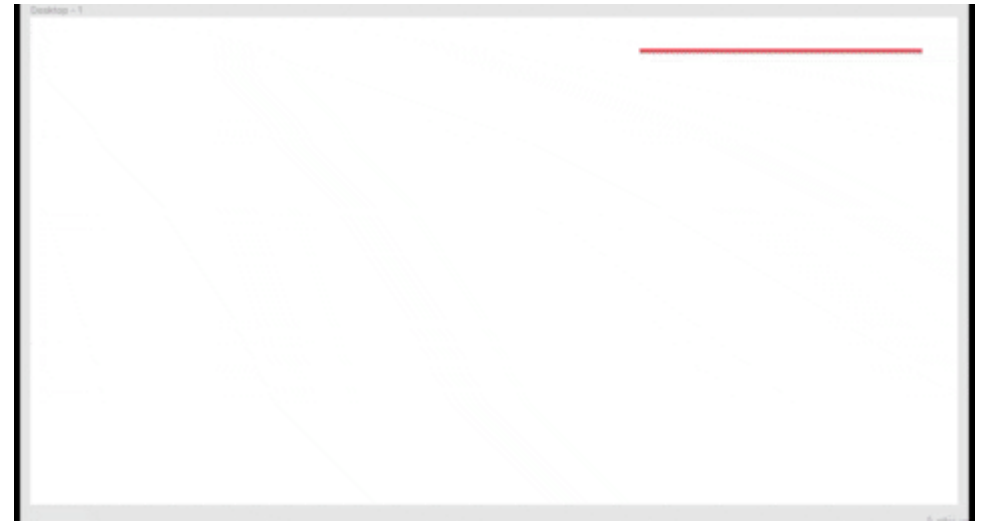
You'll see that until the first image is loaded completely, the second image doesn't start loading.

Well, JavaScript is by default Synchronous **[single threaded]**



Asynchronous System -

In this system, tasks are completed independently.

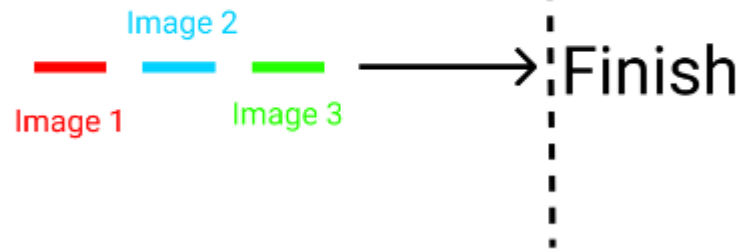


To Summarize Synchronous vs Asynchronous JS:

When three images are on a marathon, in a:

- Synchronous** system, three images are in the same lane. One can't overtake the other. The race is finished one by one. If image number 2 stops, the following image stops.

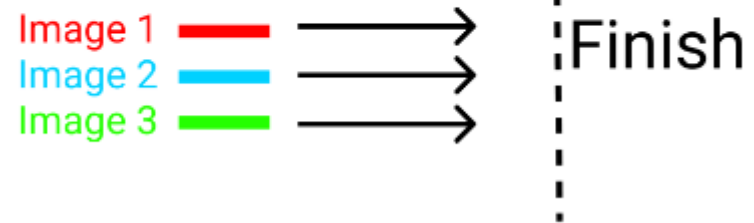
Synchronous



To Summarize Synchronous vs Asynchronous JS:

- **Asynchronous system**, the three images are in different lanes. They'll finish the race on their own pace. Nobody stops for anybody:

Asynchronous



Synchronous System code example -

```
console.log("I ");  
  
console.log(" eat ");  
  
console.log(" Ice Cream ");
```


Asynchronous System code example -

Let's say it takes two seconds to eat some ice cream. Now, let's test out an asynchronous

```
console.log("Start")
setTimeout(function() {
  console.log("Hey I am good");
}, 3000)
console.log("End")
```

```
console.log("I");
```

```
// This will be shown after 2 seconds
```

```
setTimeout(
  ()=>
  {
    console.log("eat");
  }, 2000)
```

```
console.log("Ice Cream")
```

What are callbacks in JavaScript?

When doing a complex task, we break that task down into smaller steps. To help us establish a relationship between these steps according to time (optional) and order, we use callbacks.

Callback illustrated

```
Function One (){  
    // Do something  
}  
Function Two (call_One){  
    // Do something else  
    call_One()  
}
```

Two(One); ← code is being executed

An example of a callback

What are callbacks in JavaScript?

A **callback function** is a function passed into another function as an argument, which is then invoked inside the outer function to complete some kind of routine or action.

There are two ways in which the callback may be called: *synchronous* and *asynchronous*. Synchronous callbacks are called immediately after the invocation of the outer function, with no intervening asynchronous tasks, while asynchronous callbacks are called at some point later, after an asynchronous operation has completed.

Normal function:-

```
// function
function greet(name) {
    console.log('Hi' + ' ' + name);
}

greet('Peter'); // Hi Peter
```

Callback function

```
// function
function greet(name, callback) {
    console.log('Hi' + ' ' + name);
    callback();
}

// callback function
function callMe() {
    console.log('I am callback function');
}

// passing function as an argument
greet('Peter', callMe);
```

Callback function example

Example: Program with setTimeout()

```
// program that shows the delay in execution

function greet() {
    console.log('Hello world');
}

function sayName(name) {
    console.log('Hello' + ' ' + name);
}

// calling the function
setTimeout(greet, 2000);
sayName('John');
```

Example: Using a Callback Function

```
// Callback Function Example
function greet(name, myFunction) {
    console.log('Hello world');

    // callback function
    // executed only after the greet() is executed
    myFunction(name);
}

// callback function
function sayName(name) {
    console.log('Hello' + ' ' + name);
}

// calling the function after 2 seconds
setTimeout(greet, 2000, 'John', sayName);
```

Note: The callback function is helpful when you have to wait for a result that takes time. For example, the data coming from a server because it takes time for data to arrive.

Disadvantages of callback-

Callback Hell

- Callback hell is a phenomenon where a Callback is called inside another Callback. It is the nesting of multiple Callbacks inside a function. If you look at the design of the code, it seems just like a pyramid. Thus the Callback hell is also referred to as the **‘Pyramid of Doom’**.
- Callback hell structurally is just a nesting of function calls inside a function. But, it becomes difficult to understand and keep track of the nesting once the size of the nesting is increased.

*We can avoid this problem using a few methods like **Promises**, **async/await**, and Partitioning bigger functions into smaller ones.*

```
function print(i){
    window.alert("This is call number "+i);
}

function fun1(callback){
    setTimeout(()=>{

        let i = 1 ;
        callback(i); i++ ;
        setTimeout(()=>{

            callback(i); i++;
            setTimeout(()=>{

                callback(i); i++ ;
                setTimeout(()=>{

                    callback(i); i++ ;
                    // .... and so on

                }, 800)
            }, 700)
        }, 500)
    }, 300)
    }, 100)
}

// Calling fun1 with print function as parameter
fun1(print);
```

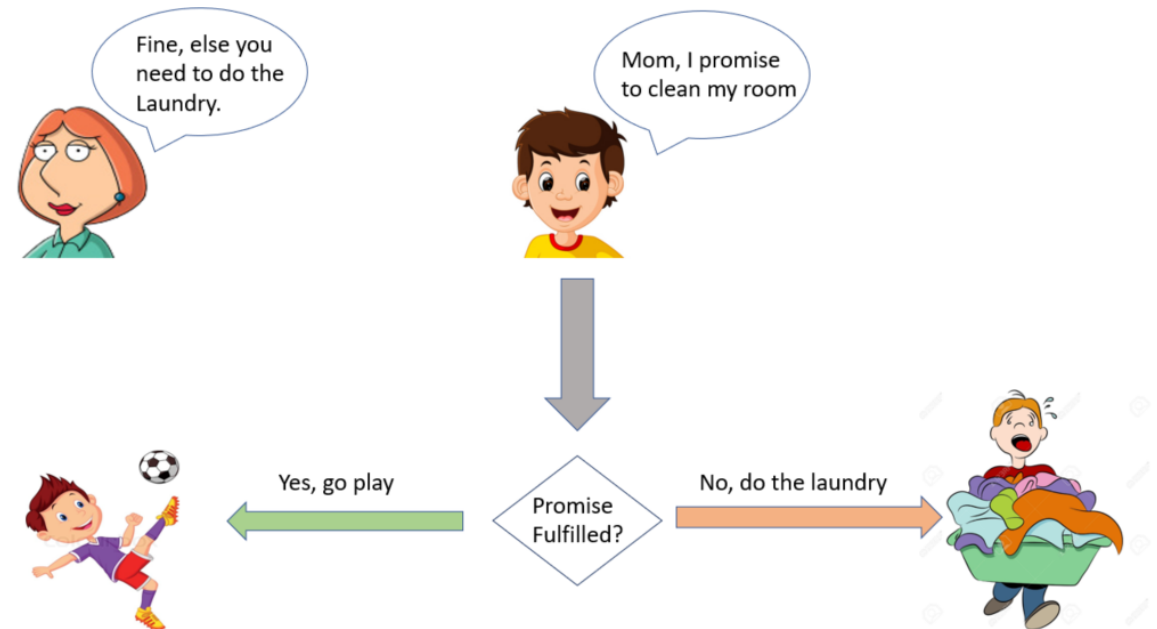
Disadvantages of callback-

Inversion of Control - In the context of callbacks, inversion of control is the notion of having code under your control in one part of the program, then handing control over to a callback in another part of the program.

```
const cart=["Shoes","shirts","trousers"];  
api.createOrder(cart,function()  
{  
  api.proceedToPayment();  
});
```

JavaScript Promises

- Javascript is a **synchronous** programming language where instructions are blocked until the current task is completed. But there might be cases where we don't want the program execution to wait, like making an API(Application Programming Interface) call to fetch data. In these cases where **asynchronous operations** have to be performed, we use **Callback, Promises, and async-await**.
- Before JavaScript promises were introduced, callbacks were used to perform asynchronous operations. But over the years, JavaScript promises replaced callback functions and are now considered the effective method to handle asynchronous operations.



JavaScript Promises

- A Javascript promise is similar to a promise in real life. When we make a promise in real life, it guarantees that we will do something in the future. A promise has two possible outcomes: it will be kept when the time comes or won't. The same is true for promises in JavaScript. When a JavaScript promise is defined, it will be resolved when the time comes or rejected.
- JavaScript Promise are easy to manage when dealing with multiple asynchronous operations where callbacks can create callback hell leading to unmanageable code. Prior to promises events and callback functions were used but they had limited functionalities and created unmanageable code. Multiple callback functions would create callback hell that leads to unmanageable code. Promises are used to handle asynchronous operations in JavaScript.

Syntax

Parameters:

- The promise constructor takes only one argument which is a callback function
- The callback function takes two arguments, resolve and reject
- Perform operations inside the callback function and if everything went well then call resolve.
- If desired operations do not go well then call reject.

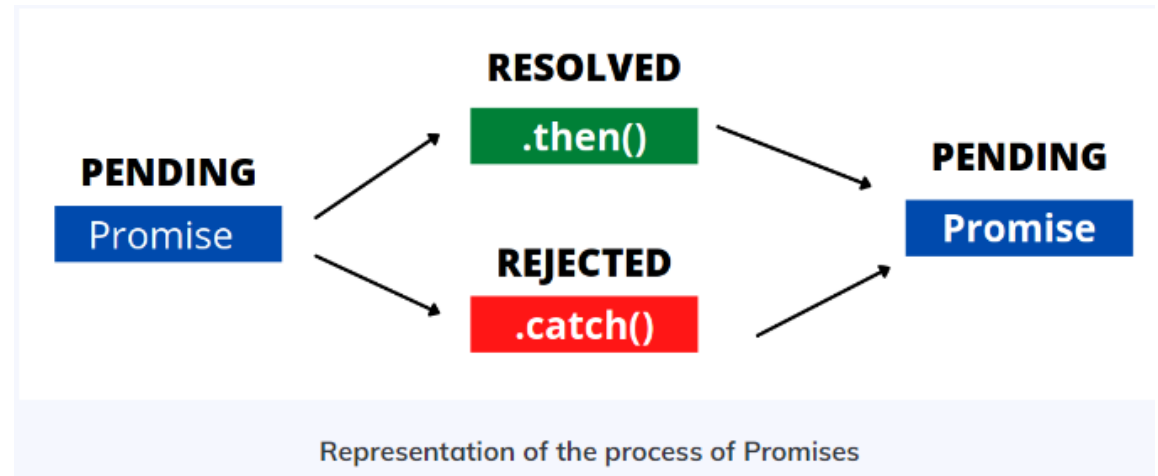
```
let promise = new Promise(function(resolve, reject){  
    //do something  
});
```

Working of promises

- The JavaScript Promise object supports two properties: **state** and **result**.

For example, if we request some data from the server using a Promise, it will be pending until the data is received. When we receive the data from the server successfully, the promise will be in the resolved state. If we don't receive any data, the promise will be in the rejected state.

If there are multiple requests, after the first promise is resolved or rejected, a new process is attached to it directly. This is called chaining.



States of promises

- 1.fulfilled:** Action related to the promise succeeded
- 2.rejected:** Action related to the promise failed
- 3.pending:** Promise is still pending i.e. not fulfilled or rejected yet
- 4.settled:** Promise has been fulfilled or rejected

promises example

```
var p = new Promise((resolve,reject)=>
{
    var name="Anil";
    if(name=="Anil")
    {
        resolve(name);
    }
    else
    {
        reject("name does not match");
    }
});
p.
then((name)=>
{
    console.log(name);
})

.catch((err)=>
{
    console.log(err);
});
```

Below code gives the output as pending since API is not loaded yet

```
const axios = require("axios"); 18.3K (gzipped: 6.8K)

const data = axios.get("https://cat-fact.herokuapp.com/facts");
console.log(data);
```

- Real time use of Promises

Below code work perfectly

```
const axios = require("axios"); 18.3K (gzipped: 6.8K)

const data = axios.get("https://cat-fact.herokuapp.com/facts");

data
  .then((res) => {
    console.log(res.data);
  })
  .catch((err) => {
    console.log(err);
  });
```

Promise consumers

Promises can be consumed by registering functions using **.then** and **.catch** methods.

1.Promise then() Method: It is invoked when a promise is either resolved or rejected. It may also be defined as a carrier that takes data from promise and further executes it successfully.

Parameters: It takes two functions as parameters.

- The first function is executed if the promise is resolved and a result is received.
- The second function is executed if the promise is rejected and an error is received. (It is optional and there is a better way to handle error using *.catch() method*)

Syntax:

```
.then(function(result){  
    //handle success  
}, function(error){  
    //handle error  
})
```

promises example

This example shows how the then method handles when a promise is resolved

```
let promise = new Promise(function (resolve,
reject) {
    resolve("You are a tech person");
})
```

```
promise
    .then(function (successMessage) {
        //success handler function is invoked
        console.log(successMessage);
    }, function (errorMessage) {
        console.log(errorMessage);
    });
```

promises example

This example shows the condition when a rejected promise is handled by second function of then method

```
let promise = new Promise(function (resolve, reject) {  
    reject("Promise Rejected")  
})
```

```
promise  
    .then(function (successMessage) {  
        console.log(successMessage);  
    }, function (errorMessage) {  
        //error handler function is invoked  
        console.log(errorMessage);  
    });
```


Promise consumers

2. Promise catch() Method: It is invoked when a promise is either rejected or some error has occurred in execution. It is used as an Error Handler whenever at any step there is a chance of getting an error.

Parameters: It takes one function as a parameter.

- Function to handle errors or promise rejections.(.catch() method internally calls .then(null, errorHandler), i.e. .catch() is just a shorthand for .then(null, errorHandler))

Syntax:

```
.catch(function(error){  
    //handle error  
})
```

promises example

This example shows the catch method handling the reject function of promise.

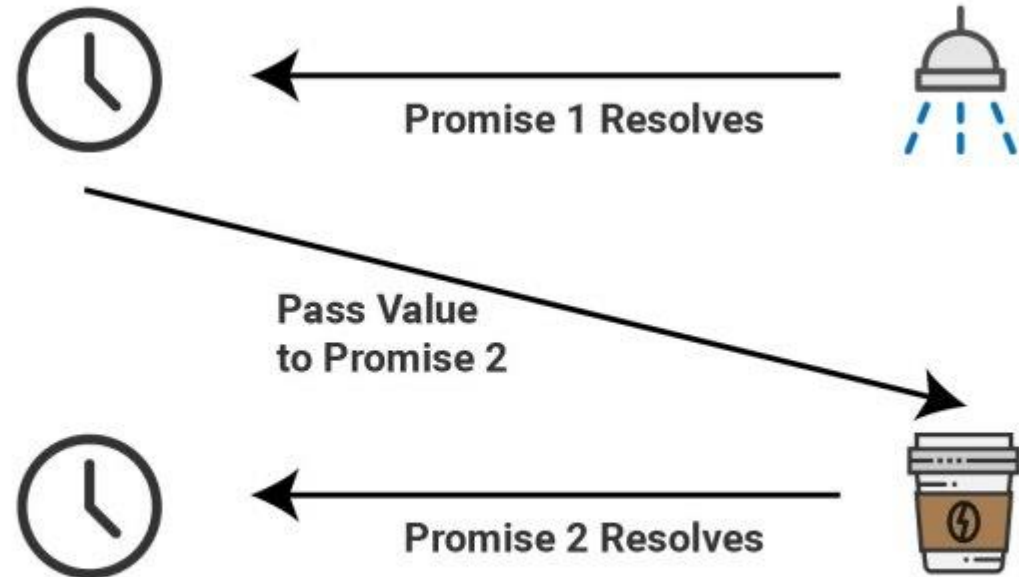
```
let promise = new Promise(function (resolve,
reject) {
    reject("Promise Rejected")
})
```

```
promise
    .then(function (successMessage) {
        console.log(successMessage);
    })
    .catch(function (errorMessage) {
        //error handler function is invoked
        console.log(errorMessage);
    });
```

async, await

- **Async** simply allows us to write **promises-based** code as if it was synchronous and it checks that we are not breaking the execution thread.
- Async functions will always return a value. It makes sure that a promise is returned and if it is not returned then JavaScript automatically wraps it in a promise which is resolved with its value.

```
const morningRoutine = async (startTime) => {
```



```
}
```

async, await

- We use the async keyword with a function to represent that the function is an asynchronous function.
- The async function returns a promise.

The syntax of async function is:

```
async function name(parameter1, parameter2, ...parameterN) {  
    // statements  
}
```

Here,

name - name of the function

parameters - parameters that are passed to the function

Async function example

```
// async function example

async function f() {
  console.log('Async function.');
  return Promise.resolve(1);
}

f();
```

Output

```
Async function.
```

Async function example

Since this function returns a promise, you can use the chaining method `then()` like this:

```
async function f() {  
  console.log('Async function.');  
  return Promise.resolve(1);  
}  
  
f().then(function(result) {  
  console.log(result)  
});
```

Output

```
Async function  
1
```

Javascript await keyword

The await keyword is used inside the async function to wait for the asynchronous operation.

The syntax to use await is:

```
let result = await promise;
```

Javascript await keyword

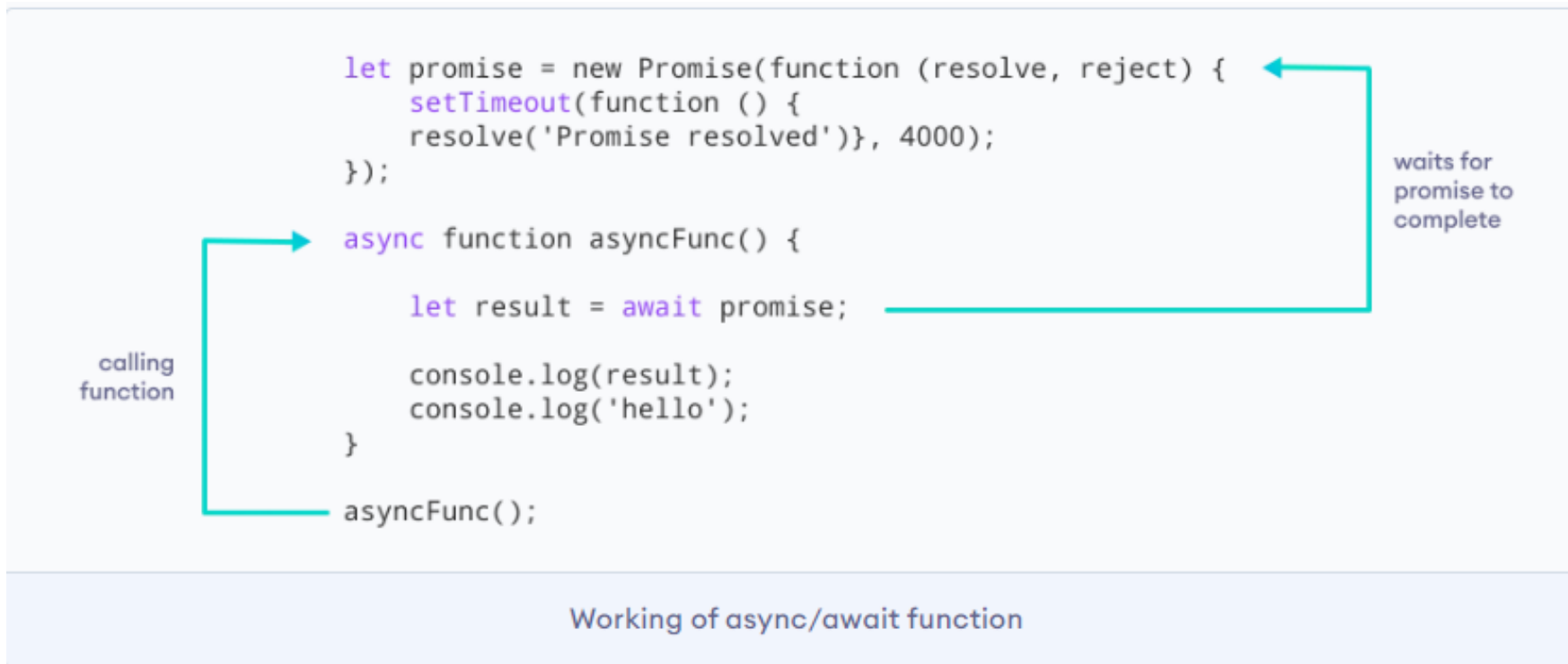
The use of await pauses the async function until the promise returns a result (resolve or reject) value. For example,

Output

```
Promise resolved  
hello
```

```
// a promise  
let promise = new Promise(function (resolve, reject) {  
  setTimeout(function () {  
    resolve('Promise resolved')}, 4000);  
});  
  
// async function  
async function asyncFunc() {  
  
  // wait until the promise resolves  
  let result = await promise;  
  
  console.log(result);  
  console.log('hello');  
}  
  
// calling the async function  
asyncFunc();
```


Javascript await keyword



Note: You can use `await` only inside of `async` functions.

Javascript await keyword

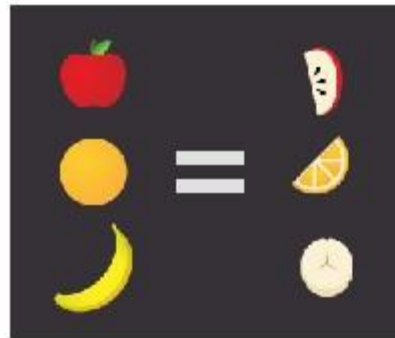
```
let promise1;  
let promise2;  
let promise3;  
  
async function asyncFunc() {  
    let result1 = await promise1;  
    let result2 = await promise2;  
    let result3 = await promise3;  
  
    console.log(result1);  
    console.log(result1);  
    console.log(result1);  
}
```

In the above program, await waits for each promise to be complete.

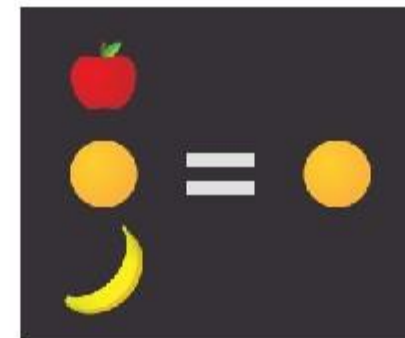
JavaScript Map, Reduce, and Filter

- Map, reduce, and filter are all array methods in JavaScript
- Each one will iterate over an array and perform a transformation or computation
- Each will return a new array based on the result of the function

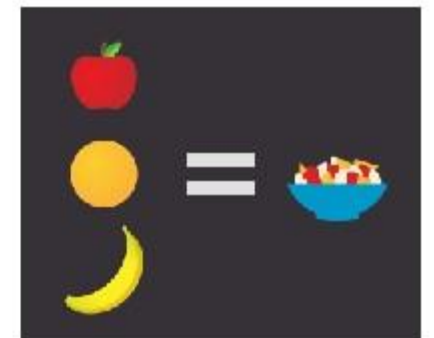
Array.map()



Array.filter()



Array.reduce()



Map

- The map() method is used for creating a new array from an existing one, applying a function to each one of the elements of the first array.

```
let arr = [45, 23, 21]
// console.log(arr)
▼ let a = arr.map((value, index, array) => {
  console.log(value, index, array)
  return value + 1
})
console.log(a)
```

Example-

```
const numbers = [1, 2, 3, 4];
const doubled = numbers.map(item =>
item * 2);
console.log(doubled); // [2, 4, 6, 8]
```

```
let arr = [45, 23, 21]
// console.log(arr)
▼ let a = arr.map((value) => {
  console.log(value)
  return value + 1
})
console.log(a)
```

Filter

- The filter() method takes each element in an array and it applies a conditional statement against it
- If this conditional returns true, the element gets pushed to the output array. If the condition returns false, the element does not get pushed to the output array.

Example-

```
const numbers = [1, 2, 3, 4];  
const evens = numbers.filter(item => item  
% 2 === 0);  
console.log(evens); // [2, 4]
```

```
// Array filter method  
let arr2 = [45, 23, 21, 0, 3, 5]  
let a2 = arr2.filter((a)=>{  
  return a<10  
})  
console.log(a2)
```

Reduce

- The reduce() method reduces an array of values down to just one value. To get the output value, it runs a reducer function on each element of the array.

Example-

```
const numbers = [1, 2, 3, 4];  
const sum = numbers.reduce(function (result, item)  
{  
  return result + item;  
}, 0);  
console.log(sum); // 10
```

```
// Array reduce method  
let arr3 = [1, 2, 3, 5, 2, 1]  
let newarr3 = arr3.reduce((h1, h2)=>{  
  return h1 + h2  
})  
console.log(newarr3)
```